



SAPIENZA
UNIVERSITÀ DI ROMA

GRIT - GOAL tRaces Injection Tool

*Progettazione e Sviluppo di una Proxy
Application per la Simulazione Trace-Driven di
Applicazioni HPC*

Facoltà di Ingegneria dell'Informazione, Informatica e
Statistica

Dipartimento di Informatica

Corso di Laurea in Informatica

Candidato:

Valerio Cola

Matricola 2060062

Relatore:

Prof. Daniele De Sensi

Anno Accademico 2025/2026

GRIT - GOAL tRaces Injection Tool

Relazione di Tirocinio, Laurea in Informatica. Sapienza Università di Roma

© 2026 Valerio Cola. Tutti i diritti riservati.

Questa relazione è stata composta con L^AT_EX.

Email dell'autore: valeriocola2003@gmail.com

Sommario

L'avvento delle architetture High Performance Computing richiede strumenti di validazione e profilazione sempre più accurati per ottimizzare l'efficienza degli applicativi paralleli. Eseguire sessioni di test e benchmarking su larga scala comporta tuttavia costi energetici proibitivi e severe limitazioni nell'accesso alle risorse. In questo contesto, le Proxy Application basate sull'esecuzione di tracce (trace-driven) offrono un'alternativa valida.

Per superare queste limitazioni, il presente elaborato propone GRIT - GOAL tRaces Injection Tool, un framework C++ scalabile e distribuito per l'iniezione dinamica di tracce MPI. GRIT agisce come uno "scheletro" comunicativo che inietta i pattern di rete e i carichi computazionali estratti da esecuzioni reali, sostituendo il massiccio calcolo scientifico con intervalli temporali modulabili.

La validazione sperimentale è stata condotta analizzando varie tracce tra cui LULESH, caratterizzato da comunicazioni locali e OpenMX, dominato da comunicazioni che saturano la banda di rete disponibile. Riducendo artificialmente i vincoli computazionali, GRIT si è dimostrato capace di isolare il traffico di rete e il carico computazionale configurandosi come uno strumento efficace per identificare i colli di bottiglia e testare le prestazioni su architetture pre-Exascale, fornendo metriche sulla scalabilità senza incorrere nei costi computazionali ed energetici di un'esecuzione fisica completa.

Indice

1	Introduzione	1
1.1	Il contesto dell'High Performance Computing e la necessità di simulazione	1
1.2	Limitazioni del testing su infrastrutture HPC reali: costi e scalabilità	1
1.3	GRIT: Framework per l'Iniezione di Tracce per la Simulazione di Ambienti MPI	3
1.4	Il Paradigma delle Proxy Applications Trace-Driven	4
1.4.1	Case study: LULESH	4
1.4.2	Case study: OpenMX	5
2	Background Tecnologico	6
2.1	Lo Standard MPI	6
2.1.1	Modello di Esecuzione e Comunicatori	6
2.1.2	Operazioni Collettive e Sincronizzazione Globale	6
2.2	Il Modello LogGOPS per le comunicazioni HPC	7
2.3	Workflow di tracciamento: dall'applicazione reale al grafo delle dipendenze	8
2.3.1	Profiling dinamico tramite liballprof	9
2.3.2	Generazione del grafo delle dipendenze con Schedgen	9
2.4	Analisi comparativa dei formati di traccia: GOAL e GOALPP	10
2.4.1	Il formato GOAL: Espansione e limiti di astrazione	10
2.4.2	Il formato GOALPP: Preservazione della semantica collettiva	12
2.4.3	Sintassi delle dipendenze e parallelismo Requires vs Irequires	12
2.4.4	Gestione dei Comunicatori e Contesti di Esecuzione	13
2.4.5	Case study: L'espansione della MPI_Allreduce	14
2.4.6	Il Grafo Aciclico Diretto (DAG)	17
2.5	Problematiche di collisione dei metadati: la gestione dei Tag Sintetici	18
2.5.1	Il limite della codifica	18
2.5.2	L'evoluzione della funzione MAKE_TAG in Schedgen	18
3	Architettura e Implementazione	20
3.1	Obiettivi di progettazione: scalabilità, asincronia e fedeltà temporale	20
3.1.1	Scalabilità e De-sincronizzazione del Controllo	20
3.1.2	Asincronia e Progresso della Simulazione	20
3.1.3	Fedeltà Temporale e Modellazione del Rumore	21

3.2	Il Parser	21
3.2.1	Algoritmo di analisi	22
3.2.2	Costruzione del Grafo delle Dipendenze (DAG)	22
3.2.3	Ottimizzazione della memoria: Allocazione Dinamica dei Payload	23
3.3	Lo Scheduler	24
3.3.1	Gestione delle Code	24
3.3.2	Logica di avanzamento asincrono e prevenzione dei Deadlock	25
3.3.3	Ottimizzazione Computazionale della Waiting Queue	29
3.3.4	Esempio di simulazione	30
3.4	Meccanismi di simulazione	31
3.4.1	Modellazione della computazione tramite Sospensione Temporale	31
3.4.2	Iniezione del rumore di sistema: il modello Gaussiano	31
3.4.3	Supporto Multi-Formato e Flessibilità dello Scheduler	33
4	Valutazione Sperimentale e Analisi Prestazionale	34
4.1	Metodologia e ambiente di test	34
4.2	Il Validator	34
4.2.1	Workflow di validazione	34
4.2.2	Analisi dell'overhead e soglie di tolleranza nel Validator	36
4.3	Analisi Comparativa: Impatto dei formati GOAL vs GOALPP	37
4.4	Studio di Scaling della Computazione	38
4.4.1	Identificazione dei colli di bottiglia: Compute-bound vs Network-bound	39
4.5	Impatto del Rumore sulla Sincronizzazione Globale	41
4.6	Analisi Prestazionale	42
4.6.1	Analisi dell'Efficienza del Modulo Parser	43
4.6.2	Profiling della Memoria e Allocazione Dinamica	44
5	Conclusioni e Sviluppi Futuri	47
5.1	Sintesi dei risultati ottenuti	47
5.2	Considerazioni sulla scalabilità del tool GRIT	47
5.3	Sviluppi Futuri: Integrazione dei Carichi AI e il Formato GOAL NCCL	48
5.3.1	Semantica e Mappatura delle Risorse nel Formato NCCL	49
5.3.2	Analisi del Tracciato ed Estensione dei Moduli di GRIT	50
	Ringraziamenti	55

Capitolo 1 - Introduzione

1.1 Il contesto dell'High Performance Computing e la necessità di simulazione

L'High Performance Computing costituisce oggi il motore tecnologico fondamentale per l'avanzamento della ricerca scientifica e industriale, permettendo la risoluzione di problemi computazionali complessi in domini che spaziano dalla fluidodinamica alla chimica quantistica, consolidandosi inoltre come infrastruttura per l'addestramento di modelli avanzati di Intelligenza Artificiale la cui ottimizzazione richiede un elevato grado di parallelismo e l'elaborazione di dataset massivi.

L'evoluzione di questi sistemi ha portato a un'adozione diffusa del paradigma della programmazione parallela. Tuttavia, come evidenziato nella roadmap dell'International Exascale Software Project [1], il raggiungimento dell'era Exascale non rappresenta esclusivamente un traguardo hardware, ma impone una revisione strutturale dell'intero ecosistema software. Dongarra et al. sottolinea che la complessità dei supercomputer moderni richiede un approccio di “co-design”, in cui lo sviluppo di strumenti di analisi e simulazione deve procedere di pari passo con la progettazione delle architetture.

In questo spazio si inserisce la sfida della gestione energetica: le frontiere dell'efficienza, esplorate recentemente da Karimi et al. [2] tramite l'analisi del Power Management, rivelano che la gestione dinamica del voltaggio e della frequenza in sistemi eterogenei, CPU e GPU, è fondamentale per evitare colli di bottiglia termici. Pertanto, la simulazione del carico di lavoro diventa uno strumento indispensabile per prevedere il consumo energetico e il comportamento termico, ma anche le prestazioni generali prima dell'allocazione fisica delle risorse. In questo scenario, la capacità di prevedere accuratamente le prestazioni di un'applicazione senza dover ricorrere all'esecuzione su cluster diventa una necessità. La simulazione si pone come strumento indispensabile per analizzare il comportamento delle applicazioni, identificare i colli di bottiglia e valutare l'efficienza degli algoritmi in scenari ipotetici o su architetture non ancora accessibili.

1.2 Limitazioni del testing su infrastrutture HPC reali: costi e scalabilità

Nonostante il progresso tecnologico, il testing e il benchmarking di applicazioni su infrastrutture HPC reali presentano limitazioni che rendono la simulazione una fase

fondamentale. La crescente complessità del software moderno richiede strumenti capaci di isolare i pattern prestazionali senza l'onere di eseguire l'intero software. In questo contesto, le Proxy Applications permettono di estrarre e riprodurre fedelmente il solo carico comunicativo e computazionale di un software massivo rendendole una soluzione preferibile in molte fasi dello sviluppo per affrontare i seguenti vincoli strutturali:

- **Costi Hardware ed Energetici:** Oltre all'hardware, che comporta costi proibitivi per l'occupazione di migliaia di nodi di calcolo per sessioni prolungate di debugging e profiling, il consumo elettrico di un sistema moderno rappresenta un vincolo che limita la libertà di sperimentazione. Ricerche condotte sui più grandi cluster al mondo [2] evidenziano come l'eterogeneità introduca variabili di consumo non lineari, rendendo ogni sessione di benchmarking su grande scala un investimento economico e ambientale significativo. A questo si aggiunge il problema del rumore di sistema, OS Jitter. Lo studio condotto sul supercomputer ASCI Q da Petrini, Kerbyson e Pakin [3] ha dimostrato come anche perturbazioni minime a livello di kernel possano causare ritardi catastrofici nelle sincronizzazioni globali, riducendo drasticamente il throughput effettivo.
- **Scalabilità e disponibilità:** L'accesso esclusivo a cluster è spesso limitato da rigide politiche di scheduling e lunghe code di attesa. Testare la scalabilità di un software su milioni di core non è un'operazione ripetibile agevolmente a causa della scarsa disponibilità di tempo macchina.
- **Analisi Predittiva:** L'impiego della simulazione abilita la conduzione di analisi predittive volte a stimare le prestazioni su architetture non ancora disponibili o configurazioni hardware ipotetiche. Attraverso la manipolazione dei parametri temporali si può valutare l'impatto di un incremento prestazionale delle unità computazionali, ad esempio, CPU o GPU raddoppiate in capacità di calcolo. Tale approccio garantisce inoltre un'elevata modularità, permettendo di isolare e simulare esclusivamente specifiche porzioni dell'applicativo. Ciò facilita l'identificazione di colli di bottiglia e lo studio della scalabilità, senza la necessità di allocare le risorse richieste per l'esecuzione dell'intero software massivo.
- **Benchmarking su larga scala:** Le applicazioni reali sono spesso composte da milioni di righe di codice, rendendo difficile isolare l'impatto di una singola modifica sulle prestazioni globali.

1.3 GRIT: Framework per l'Iniezione di Tracce per la Simulazione di Ambienti MPI

Nello scenario di crescente complessità precedentemente descritto si colloca lo sviluppo di GRIT, GOAL tRaces Injection Tool. GRIT è stato progettato come framework di iniezione di tracce, capace di convertire descrizioni statiche del comportamento di un'applicazione in programmi C++ funzionali ed eseguibili in un ambiente MPI reale.

A differenza dei simulatori a eventi discreti, che modellano il comportamento della rete tramite astrazioni puramente matematiche, GRIT opera come un "iniettore" di traffico: esso riproduce fedelmente le interazioni comunicative e le durate computazionali descritte nella traccia, interfacciandosi direttamente con il software e l'hardware del sistema di destinazione. Questo approccio permette di osservare l'interazione tra le dipendenze logiche dell'applicazione e le caratteristiche fisiche dell'interconnessione come latenza reale e congestione di rete.

L'architettura del software si articola in tre moduli principali che garantiscono scalabilità e indipendenza tra i processi:

- **Parser:** Ogni processo MPI analizza autonomamente la propria porzione di traccia, costruendo un grafo interno di dipendenze basato su istruzioni di calcolo e comunicazione.
- **Scheduler:** Costituisce il motore della simulazione; gestisce asincronamente una Ready Queue per le istruzioni eseguibili e una Waiting Queue per monitorare il progresso delle comunicazioni non bloccanti.
- **Validator:** Assicura l'accuratezza della simulazione confrontando i log di esecuzione generati da GRIT con le tracce originali, identificando eventuali overhead o discrepanze di comunicazione.

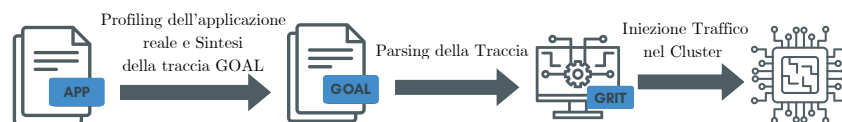


Figura 1.1: L'applicazione HPC reale viene profilata per isolare le operazioni di comunicazione e i tempi di calcolo. Il modello generato viene poi tradotto nel formato GOAL. Infine, GRIT si interfaccia con il cluster target riproducendo fedelmente il comportamento della traccia, svincolato dal reale carico computazionale

1.4 Il Paradigma delle Proxy Applications Trace-Driven

L'approccio di GRIT si fonda sul paradigma delle Proxy Applications, ovvero programmi progettati per isolare i pattern prestazionali di un software massivo. Non sono semplici riduzioni del codice sorgente, ma strumenti focalizzati sulla rappresentazione fedele del carico comunicativo e computazionale. Tale concetto si ricollega strettamente al concetto di MPI Skeletons proposto da Preissl et al. [4]. La ricerca evidenzia come la rimozione sistematica del calcolo scientifico, pur mantenendo intatta la struttura delle chiamate MPI, consenta di prevedere le prestazioni su architetture non ancora esistenti o non accessibili. Tale processo genera uno "scheletro" che preserva la struttura logica e temporale delle interazioni originarie, eliminando però l'onere computazionale e i requisiti di memoria associati ai dati scientifici, facilitando la previsione delle performance su architetture diverse.

In questo contesto, GRIT si avvale di tool di profilazione e sintesi (analizzati nella Sezione 2.3) per trasformare le Proxy Applications in scheletri comunicativi chiamati tracce GOAL (il cui formato è approfondito nella Sezione 2.4). Tale processo consente di evolvere il concetto di scheletro statico, dimostrando che la fedeltà alla struttura logica e temporale delle interazioni costituisce un predittore di performance più affidabile rispetto all'uso di test sintetici isolati. Attraverso questa astrazione, si ottiene uno strumento capace di fornire risposte puntuali sulla scalabilità dei sistemi HPC moderni, minimizzando l'impiego di risorse e massimizzando la precisione dell'analisi dei pattern di comunicazione.

1.4.1 Case study: LULESH

LULESH, Livermore Unstructured Lagrangian Explicit Shock Hydrodynamics, è formalmente classificato come una Proxy Application [5], mirata a replicare le caratteristiche prestazionali di complessi codici di simulazione su larga scala. Esso viene comunemente definito miniapp poiché si configura come una versione semplificata e compatta di un applicativo completo. Sviluppato dai Lawrence Livermore National Laboratories, il software è stato ideato per approssimare il comportamento di codici multi-fisica complessi utilizzati nella simulazione della dinamica dei fluidi [6].

LULESH risolve il problema dell'onda d'urto di Sedov utilizzando la fluidodinamica lagrangiana su una mesh esagonale. Nonostante la sua relativa semplicità, essendo composto da poche migliaia di righe di codice rispetto ai milioni dei software di produzione, esso è in grado di replicare fedelmente i pattern di accesso alla memoria e gli schemi di comunicazione MPI tipici delle applicazioni reali. L'utilizzo di tracce derivate da LULESH all'interno di GRIT offre una base di validazione solida per

l'analisi di scalabilità e fedeltà nel replicarne il comportamento, fornendo una base solida per condurre analisi predittive.

1.4.2 Case study: OpenMX

Un ulteriore Framework per la valutazione dei sistemi HPC è OpenMX (Open source package for Material eXplorer) [7], ampiamente utilizzato nella ricerca per studiare solidi, molecole e interfacce. Si tratta di un software open-source progettato per simulare la struttura elettronica dei materiali [8]. Il codice si basa sulla Density Functional Theory (DFT) un metodo quantistico per modellare sistemi complessi a molti corpi combinata con funzioni pseudo-potenziali a conservazione di norma e pseudo-atomiche [9].

Sotto il profilo architetturale, i workload generati da OpenMX differiscono radicalmente da quelli delle Proxy Application basate sulla fluidodinamica. Mentre codici come LULESH producono principalmente scambi di dati locali tra nodi adiacenti, i solutori di OpenMX impongono un traffico di rete globale. Questo comportamento deriva dalla necessità di risolvere problemi autovaloriali su matrici sparse di grandi dimensioni e di calcolare complessi operatori tridimensionali, come le Trasformate di Fourier Veloci (FFT) [10]. Queste operazioni innescano pattern di comunicazione massivi di tipo all-to-all, saturando rapidamente la banda totale disponibile nell'interconnessione.

L'esecuzione delle tracce di OpenMX tramite GRIT serve esattamente a testare questi limiti fisici. Isolando le comunicazioni dal carico computazionale, si stressano direttamente gli switch, la topologia di rete e i protocolli di routing con scambi collettivi massivi.

Capitolo 2 - Background Tecnologico

2.1 Lo Standard MPI

Il modello di programmazione principale nel calcolo ad alte prestazioni è basato sul paradigma dello scambio di messaggi. In tale ambito, lo standard è rappresentato dal Message Passing Interface (MPI) [11], una specifica di libreria che permette la comunicazione tra processi.

2.1.1 Modello di Esecuzione e Comunicatori

In un'applicazione MPI, il calcolo viene distribuito su un insieme di processi isolati che comunicano esplicitamente tramite l'invio e la ricezione di pacchetti di dati. Ogni processo è identificato da un indice numerico univoco, denominato rank, all'interno di un contesto logico chiamato comunicatore. Il comunicatore predefinito, `MPI_COMM_WORLD`, raggruppa tutti i processi avviati all'inizio dell'esecuzione [12].

Le interazioni all'interno dello standard MPI possono essere classificate in due categorie: le comunicazioni Point-to-Point e le comunicazioni collettive. Mentre le seconde coinvolgono un intero gruppo di processi, le comunicazioni Point-to-Point rappresentano l'interazione tra coppie di processi. In MPI, queste operazioni si distinguono in due modalità operative:

- **Comunicazioni Bloccanti:** Funzioni come `MPI_Send` e `MPI_Recv` sospendono l'esecuzione del processo finché l'operazione non è completata o il buffer dei dati non è reso riutilizzabile.
- **Comunicazioni Non Bloccanti:** Funzioni quali `MPI_Isend` e `MPI_Irecv` permettono di avviare l'operazione di trasferimento e restituire immediatamente il controllo al programma, favorendo la sovrapposizione tra calcolo e comunicazione. Il completamento di tali operazioni può essere verificato tramite funzioni di sincronizzazione come `MPI_Wait` o `MPI_Test`.

2.1.2 Operazioni Collettive e Sincronizzazione Globale

Le operazioni collettive coinvolgono contemporaneamente tutti i processi appartenenti a un comunicatore e sono fondamentali per la sincronizzazione globale e l'elaborazione dei dati. Le principali classi di collettive includono:

- **Sincronizzazione:** La `MPI_Barrier` impone un punto di sincronizzazione globale: nessun processo può superare tale istruzione finché tutti i membri del comunicatore non l'hanno invocata.
- **Movimento Dati:** Comprendono operazioni come `MPI_Bcast` in cui un processo invia lo stesso dato a tutti i processi, `MPI_Scatter`, utilizzata per la distribuzione di parti diverse di un buffer e `MPI_Gather`, per la raccolta di buffer verso un unico processo detto radice.
- **Riduzione Globale:** Funzioni come `MPI_Allreduce` combinano i dati provenienti da tutti i processi utilizzando operatori matematici o logici e distribuiscono il risultato finale.

L'efficienza di queste operazioni è spesso delegata alle implementazioni specifiche della libreria MPI come Open MPI o MPICH, che utilizzano algoritmi ottimizzati in base alla topologia della rete [13]. Tuttavia, tale ottimizzazione non è sempre garantita poiché le prestazioni sono sensibilmente affette da altri fattori come la dimensione dei messaggi, la saturazione dei link o il rumore del sistema.

2.2 Il Modello LogGOPS per le comunicazioni HPC

L'analisi delle prestazioni nei sistemi distribuiti richiede l'adozione di modelli capaci di astrarre la complessità delle interconnessioni di rete. Prima del consolidamento del modello LogGOPS sviluppato da Hoefer, Schneider e Lumsdaine [14], la ricerca si è basata su standard quali il LogP [15] e il LogGP [16], focalizzati rispettivamente sulla latenza e sul trasferimento di messaggi voluminosi. GRIT eredita la logica temporale del modello LogGOPS, il quale astrae le prestazioni di un sistema di rete attraverso sette parametri che permettono di descrivere sia il costo computazionale a carico del processore sia i limiti fisici della rete:

- **L - Latency:** Rappresenta l'estremo superiore del tempo impiegato da un messaggio per viaggiare attraverso la rete da un processore all'altro.
- **o - Overhead:** Indica il tempo fisso durante il quale il processore è impegnato nella trasmissione o nella ricezione di un messaggio; durante questo intervallo, la CPU non può eseguire altre operazioni.
- **g - Gap:** Definisce l'intervallo minimo di tempo che deve intercorrere tra l'invio (o la ricezione) di due messaggi consecutivi. È il reciproco della banda per messaggi di piccola dimensione.

- **G - Gap per byte:** Rappresenta il tempo aggiuntivo necessario per trasmettere ogni byte per messaggi di grandi dimensioni (reciproco della banda di picco della rete).
- **O - Overhead per byte:** Descrive l'incremento dell'overhead a carico della CPU per ogni byte aggiuntivo trasmesso, modellando il costo variabile della gestione dei buffer.
- **P - Processors:** Indica il numero totale di unità di elaborazione (processori/rank) coinvolte nella computazione.
- **S - Synchronization threshold:** Definisce la dimensione del messaggio oltre la quale il protocollo di comunicazione commuta dalla modalità eager a quella rendezvous, che richiede una sincronizzazione esplicita tra mittente e destinatario.

In sintesi, il modello LogGOPS trasforma l'astrazione della rete di comunicazione in un sistema fisico con vincoli definiti: ogni parametro rappresenta un costo temporale reale che il sistema deve "pagare" per muovere l'informazione.

È tuttavia fondamentale la distinzione tra l'uso teorico del modello e l'implementazione in GRIT. Mentre il simulatore LogGOPSim utilizza i parametri del modello per prevedere matematicamente il comportamento di un'applicazione su una rete teorica calcolandone i ritardi, GRIT opera come un iniettore di traffico. Esso legge la traccia e riproduce le interazioni comunicative tra processi interfacciandosi direttamente con lo stack MPI e l'hardware del sistema. In questo modo le prestazioni non sono frutto di una stima matematica ma derivano dall'esecuzione reale.

2.3 Workflow di tracciamento: dall'applicazione reale al grafo delle dipendenze

La trasformazione di un software HPC complesso in un piano di esecuzione avviene attraverso una pipeline definita in due fasi: il profiling e la sintesi della traccia. Questo workflow permette di generare un Grafo Aciclico Diretto, dove i nodi rappresentano le operazioni e gli archi definiscono i vincoli di precedenza logica e temporale.

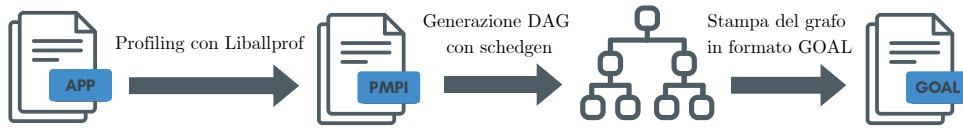


Figura 2.1: Pipeline di acquisizione e sintesi delle tracce

2.3.1 Profiling dinamico tramite liballprof

La prima fase del workflow di tracciamento prevede l'intercettazione delle comunicazioni di rete a livello MPI, operazione essenziale per estrarre il reale comportamento dell'applicativo distribuito. Tale compito è demandato a liballprof, un profiler a bassa latenza integrato nell'ecosistema di LogGOPSim.

Il meccanismo di funzionamento si basa sulla tecnica del Function Interposition, che permette di sovrapporre la libreria del profiler alle chiamate standard MPI senza richiedere la ricompilazione del codice sorgente dell'applicazione. Durante l'esecuzione, liballprof registra per ogni rank:

- La tipologia di primitiva MPI invocata.
- I parametri della comunicazione: dimensione del payload, rank target, tag etc.
- I timestamp di inizio e fine di ogni operazione campionati in microsecondi.

I dati acquisiti convergono all'interno di file testuali denominati `pmpi-trace-rank-*.txt`, dove "*" indica il numero di rank.

L'acquisizione di tracce solleva la problematica dell'intrusività del profiling. Come evidenziato da Schulz [17], l'intercettazione delle chiamate MPI può indurre latenze, innescando il cosiddetto probe effect. Tale fenomeno si manifesta come una distorsione della cronologia degli eventi alterando il comportamento originale dell'algoritmo. liballprof mitiga tale rischio limitandosi alla registrazione dei soli metadati essenziali, garantendo che l'overhead rimanga costante e non infici la validità statistica della traccia che GRIT andrà successivamente a iniettare.

2.3.2 Generazione del grafo delle dipendenze con Schedgen

I file generati dal profiler vengono successivamente elaborati dallo strumento Schedgen per la sintesi del grafo delle dipendenze. Schedgen analizza i timestamp globali e le relazioni tra le operazioni di invio e ricezione per identificare i vincoli tra le istruzioni, costruendo un Grafo Aciclico Diretto (DAG).

In questo grafo, ogni nodo rappresenta un'operazione elementare di calcolo o di comunicazione, mentre ogni arco definisce un vincolo di precedenza logica, dipendenza. Il risultato finale è un file in formato GOAL o GOALPP che elenca, per ogni rank, la sequenza di istruzioni e i relativi requisiti di esecuzione.

2.4 Analisi comparativa dei formati di traccia: GOAL e GOALPP

La fedeltà della simulazione all'interno di GRIT è determinata dalla modalità con cui le operazioni di comunicazione vengono codificate nel file di traccia. Si analizzano di seguito le due principali varianti del formato, evidenziando come la scelta della rappresentazione influenzi la capacità dello strumento di sfruttare le ottimizzazioni di rete.

2.4.1 Il formato GOAL: Espansione e limiti di astrazione

Nel formato GOAL standard, le operazioni collettive non sono presenti come entità atomiche, poiché vengono decomposte in blocchi di primitive di basso livello Point-to-Point mediante algoritmi di disseminazione. Questa frammentazione impone una sequenza di istruzioni di invio, ricezione e calcolo intermedio che simula l'algoritmo interno della collettiva.

Si consideri il seguente estratto di una traccia GOAL standard:

```
1 rank 0 {  
2     11: calc 120  
3     12: send 64b to 1 tag 100  
4     13: recv 64b from 3 tag 100  
5     ...  
6     12 requires 11  
7     13 requires 12  
8 }
```

Listing 2.1: Estratto di traccia GOAL standard

La sintassi si basa su keyword specifiche che permettono di isolare le componenti computazionali e comunicative:

- **Ambito di Esecuzione:**

- La keyword `rank [ID]` delimita il blocco di istruzioni assegnate al singolo processo.

- Tutte le operazioni e i vincoli definiti all'interno delle parentesi graffe hanno visibilità locale al rank specificato.
- **Identificatori di Istruzione:**
 - Ogni riga è preceduta da un'etichetta univoca (es. 11:, 12:), che agisce come nodo all'interno del grafo delle dipendenze.
 - Queste etichette sono fondamentali per permettere allo scheduler di referenziare e sbloccare le operazioni successive.
- **Operazioni Atomiche:**
 - **calc:** Rappresenta il tempo di occupazione della CPU per operazioni di calcolo locale. Il valore è espresso nell'unità temporale definita, tipicamente microsecondi.
 - **send / recv:** Descrivono il trasferimento di dati Point-to-Point. Includono metadati come la dimensione del payload in byte, il rank di destinazione o sorgente e l'identificativo del messaggio per il matching MPI.
- **Gestione delle Dipendenze:**
 - È la base logica del formato, la clausola **[A] requires [B]** stabilisce un arco nel grafo.
 - **Vincolo di Precedenza:** L'istruzione A entra nello stato di *ready* solo dopo il completamento definitivo di B. Questo meccanismo garantisce che GRIT preservi l'ordine logico delle operazioni (es. non inviare dati prima che il calcolo che li genera sia terminato), replicando fedelmente il comportamento dell'applicativo originale.

In questa configurazione, l'istruzione l2 invia 64 Byte al processo 1 con tag identificativo 100. Inoltre necessita che l1 termini prima di poter essere eseguita grazie al primo vincolo **requires**.

Il limite principale di tale approccio risiede nella perdita di astrazione: la libreria MPI sottostante non riceve l'informazione di un'operazione globale, ma viene sollecitata da una serie di chiamate indipendenti generate dagli algoritmi di Schedgen che possono differire da quelli utilizzati da Open MPI o MPICH.

2.4.2 Il formato GOALPP: Preservazione della semantica collettiva

Il formato GOALPP, o GOAL Plus Plus, è stato introdotto per superare tali limitazioni, mantenendo la semantica delle chiamate di gruppo. In questo modo, GRIT può delegare la gestione dei flussi direttamente alla libreria MPI a runtime.

Si consideri il seguente estratto di una traccia GOALPP:

```
1 rank 0 {  
2     ...  
3     l12: bcast 64b root 0  
4     l13: bcast 40b root 2  
5     l14: allgather 8b  
6     ...  
7     l12 requires l11  
8     l13 requires l12  
9     l14 requires l13  
10 }
```

Listing 2.2: Estratto di traccia GOALPP

In questo frammento, le istruzioni l12, l13 e l14 rappresentano operazioni atomiche di alto livello. GRIT interpreta queste linee invocando direttamente le funzioni `MPI_Bcast` e `MPI_Allgather`. Tale approccio garantisce che la Proxy Application possa beneficiare delle implementazioni ottimali, come algoritmi a albero o di disseminazione, previste dallo standard MPI per la specifica architettura di rete.

2.4.3 Sintassi delle dipendenze e parallelismo Requires vs Irequires

La struttura del grafo di esecuzione è determinata dalle relazioni di precedenza che definiscono l'ordine delle operazioni. Sono presenti due tipologie di dipendenze per gestire l'avanzamento della simulazione:

- **Requires (Dependency on Completion):** Definisce un vincolo di sincronizzazione forte. Un'istruzione vincolata da questa clausola può essere eseguita solo dopo che l'operazione genitrice ha completato interamente la propria esecuzione. Questa logica è essenziale per modellare dipendenze di dato strette o la sequenzialità tra calcoli e comunicazioni bloccanti.

- **Irequires (Dependency on Initiation):** Specifica una dipendenza legata esclusivamente all'avvio dell'istruzione. È un meccanismo per abilitare la sovrapposizione tra calcolo e comunicazione, lo scheduler sblocca le istruzioni non appena la primitiva viene invocata, senza attendere il segnale di completamento.

Si consideri il seguente estratto di una traccia:

```
1 rank 0 {  
2   l1: send 1024b to 1 tag 100  
3   l2: calc 500  
4   l3: calc 100  
5   ...  
6   l2 irequires l1  
7   l3 requires l1  
8 }
```

Listing 2.3: Esempio di dipendenze Requires vs Irequires

Analizzando l'estratto, si distinguono due flussi logici basati sulla gestione della medesima operazione l1.

L'istruzione l2 di calcolo viene sbloccata non appena la send è stata inizializzata. Questo permette alla CPU di eseguire il calcolo mentre il controller di rete gestisce il trasferimento simulando il comportamento di una primitiva non bloccante.

L'istruzione l3 può iniziare solo quando l1 è stata completata. In questo contesto, requires garantisce che le risorse associate a l1 siano nuovamente disponibili o che il dato sia stato trasmesso correttamente prima di eseguire l3.

2.4.4 Gestione dei Comunicatori e Contesti di Esecuzione

L'introduzione di primitive collettive native nel formato GOALPP solleva la questione della gestione dei comunicatori multipli. Mentre l'espansione in Point-to-Point riconduce ogni interazione a rank globali, l'utilizzo di chiamate come allgather richiede la definizione del contesto di comunicazione.

In scenari caratterizzati dalla presenza di più comunicatori, si presentano le seguenti criticità tecniche:

- **Ambiguità del Rank:** In MPI, un processo possiede un rank differente per ogni comunicatore di cui fa parte. Senza un identificatore del comunicatore nella traccia, lo scheduler non è in grado di determinare quali processi debbano partecipare alla collettiva.

- **Rischio di Deadlock:** Se due gruppi di processi invocano collettive differenti (o la medesima collettiva su comunicatori diversi) e lo scheduler non distingue i contesti, si verificherebbe un disallineamento nelle barriere implicite, portando allo stallo della simulazione.

Attualmente, il framework assume che ogni operazione collettiva definita in GOALPP avvenga all'interno del comunicatore globale (MPI_COMM_WORLD). Questa semplificazione è valida per Proxy Application lineari, ma richiederebbe l'aggiunta di un nuovo parametro nel formato di traccia per supportare applicativi multi-livello complessi.

2.4.5 Case study: L'espansione della MPI_Allreduce

Per comprendere appieno la divergenza tra i formati GOAL e GOALPP, si analizzi l'operazione MPI_Allreduce. Mentre nel formato GOALPP tale operazione viene mantenuta come istruzione atomica, nel formato GOAL Schedgen la decompone seguendo un algoritmo di disseminazione.

```

1 void create_dissemination_rank(Goal *goal, int comm_rank,
  int comm_size, int datasize) {
2     int dist = 1;
3     Goal::t_id recv = -1, send = -1;
4     while (dist < comm_size) {
5         // Aggiunta istruzione al grafo
6         send = goal->Send(datasize, (comm_rank + dist) %
comm_size);
7         if (recv != -1) goal->Requires(send, recv);
8         // Aggiunta istruzione al grafo
9         recv = goal->Recv(datasize, (comm_rank - dist +
comm_size) % comm_size);
10        dist <= 1;
11    }
12 }
```

Listing 2.4: Algoritmo di Disseminazione

L'obiettivo dell'algoritmo è garantire che, al termine dell'esecuzione, ogni processo partecipante possieda il risultato globale dell'operazione. Il concetto fondamentale risiede nell'esecuzione di una serie di passi logaritmici: in ogni iterazione k , ciascun processo scambia i propri dati, che includono i contributi parziali accumulati nei passi precedenti, con un partner situato a una distanza relativa pari a 2^k . Questa

distanza raddoppia a ogni passo, permettendo la propagazione dell'informazione a tutti gli N processi in un numero di step pari a $\log_2 N$.

La variabile `dist`, inizializzata a 1, rappresenta la "distanza" logica tra i processi coinvolti nello scambio in una determinata iterazione. Il ciclo `while` prosegue finché tale distanza è inferiore alla dimensione del comunicatore. Al termine di ogni iterazione, l'operazione di bit-shift a sinistra `dist «= 1` raddoppia il valore della distanza.

L'utilizzo dell'operatore modulo `%` nelle funzioni di invio e ricezione permette di modellare la rete come un anello logico unidirezionale. Nello specifico:

- `(comm_rank + dist) (mod comm_size)` identifica il target verso cui inviare i dati.
- `(comm_rank - dist + comm_size) (mod comm_size)` calcola il rank dal quale attendere il contributo parziale. Questa aritmetica modulare assicura che, indipendentemente dalla distanza di salto, l'identificativo del rank rimanga sempre all'interno del range valido $[0, \text{comm_size} - 1]$.

Le chiamate a `goal->Send` e `goal->Recv` agiscono come costruttori di nodi all'interno del DAG. Ogni chiamata restituisce un identificativo univoco (`t_id`) che viene utilizzato per stabilire le dipendenze tramite il metodo `goal->Requires`. Essa impone che la send del passo corrente possa essere abilitata solo dopo il completamento della recv del passo precedente. Questo garantisce che i dati inviati contengano sempre l'aggregazione aggiornata dei contributi ricevuti fino a quel momento, preservando la correttezza semantica della riduzione globale.

Analisi scenario a 4 processi

Per illustrare il funzionamento logico, si consideri uno scenario con 4 processi. L'evoluzione dello stato del Rank 0 segue la progressione descritta di seguito:

Step 1: Distanza $2^0 = 1$ In questa fase iniziale, la distanza di comunicazione è unitaria:

- **Invio:** Il Rank 0 invia il proprio contributo locale al Rank 1 ($0 + 1$).
- **Ricezione:** Contemporaneamente, riceve il contributo parziale dal Rank 3 ($0 - 1 \pmod{4}$).

Al termine di questo step, il Rank 0 ha combinato il proprio dato con quello del Rank 3. Parallelamente, tutti gli altri nodi hanno effettuato scambi analoghi con i rispettivi vicini a distanza 1.

Step 2: Distanza $2^1 = 2$ Nel secondo passo, la distanza di comunicazione raddoppia. In questa fase l'istruzione di invio del Rank 0 è vincolata al completamento della ricezione dello Step 1.

- **Invio:** Una volta ottenuto il dato dal Rank 3, il Rank 0 invia il nuovo pacchetto aggregato $(0+3)$ al Rank 2 $(0 + 2)$.
- **Ricezione:** Il Rank 0 riceve dal Rank 2 il pacchetto che quest'ultimo ha aggregato nel passo precedente, ovvero l'unione dei dati di Rank 2 e Rank 1.

Grazie a questa struttura, al termine dello Step 2, il Rank 0 ha accumulato i contributi di tutti i processi della rete $(0+3+2+1)$.

Il limite dell'espansione statica

Nonostante l'impiego di un algoritmo a complessità logaritmica $O(\log n)$, l'espansione in formato GOAL presenta una criticità strutturale: essa congela la strategia comunicativa nel momento della generazione della traccia. Questo comporta due limitazioni principali:

- **Rigidità Topologica:** L'algoritmo di disseminazione scelto da Schedgen potrebbe non coincidere con quello ottimale per la specifica architettura di rete su cui GRIT viene eseguito. Ad esempio un algoritmo ad anello potrebbe essere preferibile per messaggi di grandi dimensioni.
- **Perdita di Offloading:** Molte moderne interconnessioni HPC offrono il supporto all'Hardware Collective Offloading, permettendo alla scheda di rete di gestire la riduzione senza impegnare la CPU. L'espansione in singole send/recv impedisce a GRIT di sfruttare tali accelerazioni hardware.

In GOALPP, al contrario, GRIT invoca direttamente la primitiva `MPI_Allreduce`. Delegare l'efficienza delle comunicazioni alla libreria MPI a runtime permette di beneficiare delle ottimizzazioni dinamiche descritte da Thakur, Rabenseifner e Gropp [13]. La ricerca condotta nell'ambito di MPICH ha mostrato che non esiste un algoritmo universale: la scelta tra pattern ad anello, ad albero o a raddoppio ricorsivo dipende drasticamente dalla dimensione del messaggio e dal numero di processi. Supportando il formato GOALPP, GRIT evita di congelare l'applicazione in una singola strategia, garantendo la portabilità delle prestazioni tra diverse implementazioni MPI.

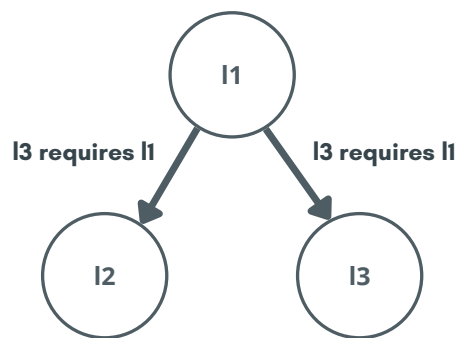
2.4.6 Il Grafo Aciclico Diretto (DAG)

Il principio cardine di GRIT risiede nell'interpretazione della traccia non come un elenco lineare di comandi, ma come un Grafo Aciclico Diretto (DAG).

Un DAG è una struttura matematica definita da:

- **Vertici:** Rappresentano le singole unità di lavoro: CALC, SEND, RECV, ecc.
- **Archi Diretti:** Rappresentano i vincoli di dipendenza *requires* e *irequires*.
- **Proprietà Aciclica:** Garantisce l'assenza di percorsi detti *loop*, riflettendo la natura unidirezionale del tempo di esecuzione; un'istruzione non può dipendere, direttamente o indirettamente, dal proprio completamento futuro.

Di seguito è riportato un esempio semplificato di DAG derivato dalla traccia Listato 2.3:



La modellazione della traccia come un DAG non è una scelta implementativa, ma poggia su solide basi matematiche. Secondo la formalizzazione di Bang-Jensen e Gutin [18], la proprietà di aciclicità è l'unico requisito che garantisce l'esistenza di un ordinamento topologico, il quale nel contesto HPC riflette la causalità temporale degli eventi. In GRIT, l'assenza di cicli assicura che lo scheduler non incorra in stalli logici infiniti, permettendo di mappare ogni istruzione come un evento discreto in una timeline coerente. Questa rigorosa struttura è ciò che permette la gestione dell'asincronia senza violare la coerenza dei dati tra i rank MPI.

2.5 Problematiche di collisione dei metadati: la gestione dei Tag Sintetici

Nella fase di generazione delle tracce, la corretta identificazione delle coppie di comunicazione mittente e destinatario è un requisito fondamentale per la ricostruzione del grafo delle dipendenze. Un problema ricorrente nei sistemi HPC riguarda la gestione dei tag e dei comunicatori MPI, i cui metadati possono collidere durante la fase di profiling, portando a una rappresentazione errata del flusso dei dati.

2.5.1 Il limite della codifica

Lo strumento Schedgen adottava una logica di packing per generare un identificativo univoco che combinasse l'id del comunicatore e il tag del messaggio in uno spazio a 32 bit. La formula utilizzata prevedeva un bit-shift del comunicatore:

$$\text{ID} = (\text{comm} \ll 24) \mid \text{tag}$$

Tuttavia tale approccio è divenuto obsoleto con l'avvento delle moderne implementazioni MPI. In queste librerie, gli handle dei comunicatori possono essere puntatori a 64 bit o interi di grandi dimensioni; di conseguenza, l'operazione di scorrimento e l'impaccamento in 32 bit causano spesso una troncatura dei dati. Questo fenomeno genera collisioni, in cui coppie diverse collassano nello stesso identificativo sintetico, rendendo impossibile per il simulatore distinguere flussi di comunicazione logicamente separati.

2.5.2 L'evoluzione della funzione MAKE_TAG in Schedgen

Per risolvere le criticità legate alle collisioni e ai limiti di precisione nella gestione dei metadati, è stata implementata una nuova logica all'interno della funzione di generazione dei tag. Invece di ricorrere a operazioni bit-a-bit, si è adottato un sistema di mappatura sintetica globale.

Per eliminare le collisioni, la generazione dei tag è stata riprogettata introducendo una mappatura sintetica globale basata su tabella.

L'implementazione tecnica della funzione è la seguente:

```

1  int MAKE_TAG(int comm, int tag) {
2      // Mappatura di ogni coppia comm + tag in un tag
        sintetico.
3      static std::map<std::pair<int, int>, int> synthetic_tags;
4      static int next_id = 0;
5
6      // I tag sintetici iniziano in un range di sicurezza
        elevato.
7      const int kSyntheticStart = 100000;
8
9      // Utilizzo della coppia come chiave univoca.
10     const std::pair<int, int> key(comm, tag);
11
12     // Inserimento della chiave se mancante.
13     // Si riserva lo 0 come "non assegnato" e si genera un
        valore sintetico alla prima occorrenza.
14     int &mapped_tag = synthetic_tags[key];
15     if (mapped_tag == 0) {
16         mapped_tag = kSyntheticStart + next_id;
17         ++next_id;
18     }
19
20     return mapped_tag;
21 }
```

Listing 2.5: Funzione di generazione di Tag Sintetici per garantire l'univocità delle comunicazioni.

Il tool effettua una mappatura tramite Tabella di Lookup che associa ogni coppia univoca di comunicatore e tag a un nuovo identificativo intero. L'uso dell'operatore di accesso per riferimento `int &mapped_tag` permette di gestire in modo efficiente sia la ricerca che l'inserimento: se la coppia non è mai stata incontrata, il valore viene inizializzato a zero e successivamente aggiornato con un nuovo ID univoco.

Per evitare qualsiasi forma di sovrapposizione con i tag reali utilizzati dall'applicazione originale che solitamente occupano range numerici bassi o con i messaggi di controllo interni delle librerie MPI, i tag sintetici vengono generati a partire da un valore elevato pari a 100.000 (`kSyntheticStart`). Questo garantisce che il simulatore operi in una zona sicura, pur rimanendo entro i limiti definiti dallo standard MPI, tipicamente $2^{31} - 1$.

Capitolo 3 - Architettura e Implementazione

3.1 Obiettivi di progettazione: scalabilità, asincronia e fedeltà temporale

La progettazione di GRIT è stata guidata dalla necessità di creare uno strumento che non fosse un semplice interprete sequenziale di tracce, ma un framework di simulazione dinamico in grado di operare nelle stesse condizioni di stress di un'applicazione HPC reale. Per raggiungere tale scopo, lo sviluppo si è concentrato su tre requisiti fondamentali: la scalabilità del sistema, la gestione asincrona delle operazioni e la fedeltà temporale della riproduzione.

3.1.1 Scalabilità e De-sincronizzazione del Controllo

In ambito HPC, la scalabilità rappresenta il requisito primario per ogni strumento di analisi. Al fine di evitare colli di bottiglia, ogni processo MPI opera come un agente autonomo, ogni modulo viene eseguito in parallelo e in modo indipendente su ogni nodo. Questo design garantisce che il tempo di inizializzazione del simulatore rimanga costante o cresca minimamente all'aumentare del numero di processi, permettendo l'esecuzione di simulazioni su cluster di vaste dimensioni senza un degrado prestazionale della fase di caricamento. Inoltre, l'allocazione delle risorse di memoria è anch'essa gestita localmente per ogni rank.

3.1.2 Asincronia e Progresso della Simulazione

Un obiettivo fondamentale del progetto è stato l'eliminazione di stalli artificiali che non appartengono alla traccia originale. La fedeltà di una Proxy Application dipende dalla sua capacità di gestire l'asincronismo delle librerie MPI. GRIT adotta un motore di esecuzione basato su code di priorità che permette di sovrapporre le operazioni di calcolo alle comunicazioni di rete. Si è prestata particolare attenzione alla risoluzione dei deadlock che possono verificarsi nella simulazione di tracce complesse, specialmente nel formato GOAL standard. Attraverso l'uso combinato di primitive non bloccanti e controlli di stato periodici si garantisce che il progresso della simulazione non venga interrotto da dipendenze circolari apparenti e preservando l'ordine logico imposto dal grafo.

3.1.3 Fedeltà Temporale e Modellazione del Rumore

L'ultimo tassello della progettazione riguarda la capacità di GRIT di agire come un iniettore trasparente. La simulazione non deve solo eseguire le chiamate corrette, ma deve farlo seguendo la cadenza temporale esatta prevista dalla traccia. Ciò comporta una gestione granulare dei tempi di calcolo e la capacità di gestire i parametri di scaling per testare scenari differenti. Inoltre, per riflettere le condizioni reali dei cluster, si è implementata una logica di iniezione di rumore di sistema. Questo fenomeno, spesso trascurato nei simulatori statici, è fondamentale per comprendere come piccole variazioni temporali possano propagarsi attraverso il grafo delle dipendenze e causare ritardi significativi nelle sincronizzazioni globali [3].

3.2 Il Parser

Il modulo Parser è il componente incaricato di tradurre le tracce in una rappresentazione interna basata su oggetti C++. La sua efficienza è fondamentale, in quanto rappresenta la fase di inizializzazione che precede la simulazione vera e propria.

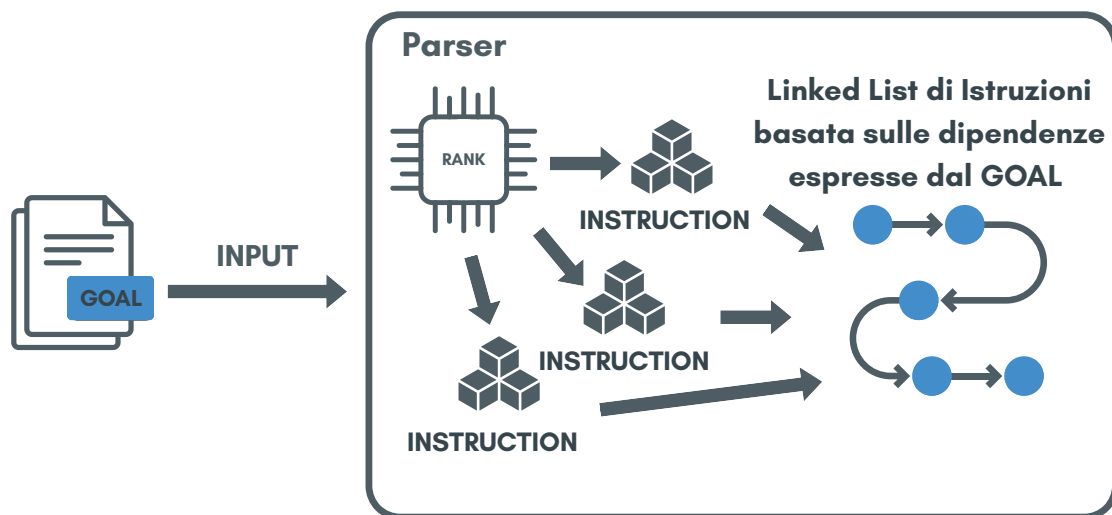


Figura 3.1: Flusso di esecuzione del modulo Parser.

3.2.1 Algoritmo di analisi

Per garantire la massima scalabilità ed evitare colli di bottiglia durante l'avvio su cluster con migliaia di nodi, GRIT adotta una strategia di parsing indipendente. Ogni processo MPI apre il file di traccia e lo analizza autonomamente. Sebbene tutti i processi leggano lo stesso file, ciascuno estrae e memorizza esclusivamente le istruzioni e le dipendenze relative al proprio identificativo.

L'algoritmo segue un processo a doppia scansione:

- **Conteggio e Allocazione:** Viene effettuata una prima scansione rapida del blocco relativo al rank per determinare il numero totale di istruzioni. Questo permette di invocare il metodo di allocazione sul vettore delle istruzioni, evitando costose riallocazioni di memoria durante la fase di popolamento.
- **Estrazione dei Metadati:** Le stringhe vengono elaborate tramite utility dedicate che rimuovono suffissi non numerici. Per ogni istruzione vengono memorizzati i relativi metadati.

3.2.2 Costruzione del Grafo delle Dipendenze (DAG)

Durante la lettura delle clausole `requires` e `irequires`, il Parser popola la struttura dati che definisce il DAG. Ogni istruzione mantiene un contatore di dipendenze mancanti, che indica quanti vincoli devono essere soddisfatti prima di poter procedere all'esecuzione. Il Parser non si limita a registrare i genitori di un'istruzione, ma costruisce attivamente le liste di adiacenza:

- **successor_ids:** Un vettore contenente gli ID delle istruzioni da sbloccare al completamento totale dell'operazione (`requires`).
- **immediate_successor_ids:** Un vettore per le istruzioni che possono essere sbloccate non appena l'operazione corrente viene avviata (`irequires`), fondamentale per la sovrapposizione tra calcolo e comunicazioni asincrone.

3.2.3 Ottimizzazione della memoria: Allocazione Dinamica dei Payload

Una delle sfide principali nella gestione di tracce massive è il consumo di RAM per ogni singolo rank. Una struttura Instruction che contenesse staticamente tutti i campi possibili per ogni tipo di operazione risulterebbe estremamente inefficiente e pedantica.

Per risolvere questo problema, si è implementata un'ottimizzazione basata sull'uso di smart pointers `unique_ptr` [19]. La struttura base dell'istruzione mantiene solo i campi comuni come ID, tipo e target. I dati extra, necessari solo per specifiche operazioni, vengono allocati dinamicamente tramite metodi on-demand, come mostrato nel seguente codice.

```

1  struct Instruction {
2      int instruction_id;
3      Type instruction_type;
4      // ... campi comuni ... //
5
6      // Allocazione on-demand tramite smart pointers
7      std::unique_ptr<SendRecvData> sendrecv_data_ptr;
8      std::unique_ptr<VectorCollectiveData> vector_data_ptr;
9      std::unique_ptr<ReductionData> reduction_data_ptr;
10
11     // Metodi di accesso
12     const SendRecvData* get_sendrecv_data() const {
13         return sendrecv_data_ptr.get();
14     }
15 };
16 // Dichiarazione della struttura
17 struct SendRecvData {
18     int send_tag = 0;
19     int recv_tag = 0;
20     // ... //
21 };

```

Listing 3.1: Struttura che descrive un'istruzione.

- **SendRecvData:** Allocatedo solo per istruzioni di tipo SENDRECV per contenere i tag e i rank sorgente e destinazione aggiuntivi.
- **VectorCollectiveData:** Utilizzato esclusivamente per operazioni collettive vettoriali, come GATHERV o ALLTOALLV, per memorizzare i vettori che

definiscono la dimensione dei singoli blocchi di dati e i rispettivi scostamenti in memoria.

- **ReductionData:** Riservato alle operazioni di riduzione per specificare l'operatore MPI richiesto.

Questa architettura, che sfrutta il polimorfismo dei dati tramite smart pointers, garantisce che la dimensione della struttura base rimanga contenuta, circa 100 byte, permettendo di caricare grafi con milioni di nodi anche su sistemi con risorse di memoria limitate per core. Al contrario, l'allocazione statica avrebbe impiegato circa 300 byte per istruzione.

Con questo approccio è possibile ridurre l'occupazione di circa il 66%, considerando ad esempio una simulazione con un grafo di 10 milioni di istruzioni per rank:

Allocazione Statica: $10^7 \times 300 \text{ byte} \approx 3 \text{ GB per rank}$

Allocazione On-demand: $10^7 \times 100 \text{ byte} \approx 1 \text{ GB per rank}$

3.3 Lo Scheduler

Lo Scheduler costituisce l'anima operativa di GRIT e agisce localmente su ogni rank per gestire l'esecuzione delle istruzioni. A differenza di un interprete sequenziale, lo Scheduler di GRIT è progettato per massimizzare il parallelismo tra le operazioni, garantendo al contempo l'assenza di stalli critici.

3.3.1 Gestione delle Code

L'avanzamento della simulazione è pilotato da due strutture dati fondamentali gestite dinamicamente:

- **Ready Queue:** Una coda che ospita gli identificativi delle istruzioni il cui numero di vincoli di dipendenza è sceso a zero. All'inizializzazione, tutte le istruzioni prive di prerequisiti vengono inserite in questa coda per avviare la simulazione.
- **Waiting Queue:** Un vettore dinamico per il tracciamento e monitoraggio continuo di tutte le operazioni di rete non bloccanti che sono state inizializzate, ma la cui finalizzazione risulta ancora pendente. Ogni record memorizzato all'interno di questa lista associa univocamente l'identificativo del task al rispettivo handle di tracciamento nativo dello stack software (`MPI_Request`),

mantenendo al contempo il riferimento al buffer di memoria allocato per il trasferimento dei dati.

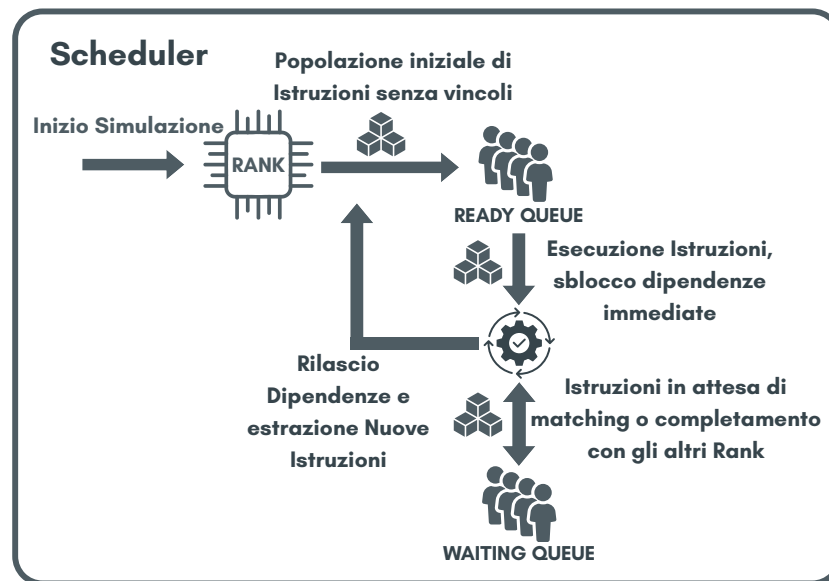


Figura 3.2: Flusso di esecuzione del Modulo Scheduler.

3.3.2 Logica di avanzamento asincrono e prevenzione dei Deadlock

Il progresso della simulazione a runtime è governato da un ciclo di controllo principale, la cui terminazione è subordinata allo svuotamento congiunto della coda dei task pronti `my_ready_queue` e della struttura adibita al monitoraggio delle comunicazioni asincrone pendenti.

Nello specifico, la gestione dei flussi non bloccanti si appoggia interamente su un vettore dinamico locale denominato `active_comms`. Mentre la `my_ready_queue` ospita gli identificativi numerici delle sole istruzioni i cui vincoli logici sono scesi a zero, `active_comms` opera come una vera e propria Waiting Queue. Essa memorizza elementi di tipo `ActiveCommunicationPP`, una struttura dati specificamente ingegnerizzata per incapsulare i metadati a basso livello necessari allo stack MPI per finalizzare i trasferimenti in background.

L'interazione tra queste strutture è espressa dal seguente algoritmo di controllo centrale del modulo di scheduling:

```

1  while (!my_ready_queue.empty() || !active_comms.empty()) {
2
3      // Fase 1: Svuotamento della coda dei task pronti (
4      // Avanzamento Computazionale)
5      while (!my_ready_queue.empty()) {
6          int current_task = my_ready_queue.front();
7          my_ready_queue.pop_front();
8
9          // Esecuzione dell'istruzione e sblocco immediato
10         // via irequires se asincrona
11         execute_instruction(current_task, my_ready_queue,
12         active_comms);
13     }
14
15     // Fase 2: Gestione e polling delle comunicazioni
16     // asincrone pendenti
17     if (!active_comms.empty()) {
18         if (my_ready_queue.empty()) {
19             // Stato di stallo computazionale: attesa
20             // bloccante opportunistica
21             MPI_Waitany(...);
22         } else {
23             // Presenza di task computazionali: verifica
24             // non bloccante dello stato della rete
25             MPI_Test(...);
26         }
27     }
28 }

```

Listing 3.2: Loop centrale dello Scheduler.

Struttura e Ruolo di active_comms

Nelle architetture parallele moderne, l'invocazione di primitive asincrone restituisce immediatamente il controllo all'applicazione chiamante prima che il trasferimento dei dati sia effettivamente completato. Tale asincronia espone il sistema a gravi rischi di corruzione della memoria e violazione della consistenza qualora i buffer dedicati vengano alterati o deallocati prematuramente dallo scheduler. Per garantire l'integrità dei dati senza introdurre barriere di sincronizzazione artificiali, GRIT incapsula i descrittori di rete all'interno della seguente struttura memorizzata in

active_comms:

```

1 struct ActiveCommunicationPP {
2     int task_id;           // Identificativo univoco nel
    DAG locale
3     MPI_Request request;   // Handle MPI per il
    tracciamento dell'operazione asincrona
4     void* buffer;         // Puntatore al buffer di
    memoria volatile allocato dinamicamente
5     double start_time;    // Timestamp di inizio dell'
    operazione (rilevato tramite MPI_Wtime)
6 };

```

Listing 3.3: Struttura per le comunicazioni attive.

Il vettore `ActiveCommunicationPP` assolve quindi a tre funzioni fondamentali nell'ecosistema di GRIT:

- Preserva il puntatore `buffer`, impedendo il rilascio della memoria volatile fino a quando l'interfaccia di rete (NIC) non abbia concluso lo svuotamento o il popolamento dei canali hardware.
- Aggrega gli handle `MPI_Request` per consentire controlli di stato multipli e ottimizzati tramite le routine native dello standard Message Passing Interface.
- Conserva il parametro `start_time` al fine di calcolare per sottrazione l'esatta durata della comunicazione a runtime, isolando l'overhead di rete dalle metriche di puro calcolo computazionale (CALC).

Avanzamento Asincrono e Politiche di Polling

La ramificazione condizionale della Fase 2 del ciclo principale descrive l'approccio adattivo di GRIT per massimizzare il throughput computazionale e prevenire fenomeni di deadlock.

- **Scenario di Avanzamento - Ready Queue non vuota:** Qualora vi siano task pronti per l'esecuzione, lo Scheduler non può permettersi di bloccare il progresso della simulazione in attesa della rete. Viene pertanto applicato un meccanismo di polling non bloccante tramite la primitiva `MPI_Test`. Questo consente di verificare lo stato di avanzamento dei canali asincroni in background; se un'operazione risulta conclusa, le dipendenze di tipo `requires` ad essa collegate vengono immediatamente decrementate, inserendo eventuali nuovi task pronti in coda senza interrompere il flusso di calcolo locale.

- **Scenario di Stallo - Ready Queue vuota:** Se il processo locale ha esaurito i task computazionali pronti ed è in attesa esclusiva di eventi di rete per sbloccare i successori nel grafo, l'attivazione di un ciclo di polling comporterebbe un inutile spreco di risorse computazionali e overhead sul bus di sistema. In questa esatta circostanza, lo Scheduler intercetta lo stallo e invoca la primitiva `MPI_Waitany`. Questa chiamata sospende il core del processore in modo efficiente, riattivando il rank non appena almeno un trasferimento asincrono viene finalizzato dalla rete.

Gestione delle Dipendenze e Flusso delle Istruzioni

La robustezza del modello unificato di GRIT si riflette nella capacità di mappare e ordinare correttamente i flussi informativi nel DAG attraverso la combinazione delle relazioni di dipendenza. Si identificano tre scenari all'interno delle tracce:

- **Istruzioni Non Bloccanti Multiple:** In presenza di più operazioni asincrone consecutive, l'architettura sfrutta il fatto che tali istruzioni rilasciano i propri successori immediati tramite archi `irequires` non appena la primitiva viene inizializzata. Lo Scheduler provvede dunque a un inserimento sequenziale immediato nel vettore `active_comms` tramite operazioni di `push_back`, consentendo la sovrapposizione in background di molteplici canali di comunicazione paralleli.
- **Istruzioni Bloccanti Consecutive:** Se la traccia specifica una sequenza di operazioni strettamente sincrone, la corretta scomposizione del grafo impone un vincolo di dipendenza forte (`requires`) tra l'istruzione precedente e la successiva. L'arco viene risolto e il contatore delle dipendenze decrementato esclusivamente al termine reale dell'operazione.
- **Scenario Misto:** Nel caso in cui un'istruzione `ISEND` preceda una `RECV` bloccante, lo Scheduler inizializza la prima, inserisce il metatesto in `active_comms` e, grazie all'arco `irequires`, passa istantaneamente all'esecuzione della `RECV`. Quest'ultima, essendo bloccante, congela il progresso logico della sua specifica linea del DAG fino a ricezione avvenuta e permette alle comunicazioni precedentemente iniettate in background di progredire.

Attraverso questa architettura, GRIT è in grado di gestire sia l'espansione delle collettive del formato GOAL sia la semantica di alto livello del formato GOALPP, delegando l'efficienza delle comunicazioni allo stack MPI pur mantenendo il controllo rigoroso sulle dipendenze applicative.

3.3.3 Ottimizzazione Computazionale della Waiting Queue

Un fattore determinante per le prestazioni del sistema è l'efficienza di rimozione dei task completati dal vettore `active_comms`. Un approccio basato sullo scorrimento degli elementi e l'invocazione della funzione standard `erase()` comporterebbe una complessità computazionale lineare pari a $O(n)$, dove n rappresenta il numero di comunicazioni simultaneamente attive. All'aumentare della scala della simulazione o in presenza di tracce dense, tale overhead degraderebbe le prestazioni del simulatore stesso.

Poiché l'ordine dei descrittori all'interno della Waiting Queue non inficia la correttezza logica della simulazione, GRIT implementa una tecnica di rimozione ad alta efficienza in tempo costante $O(1)$:

```

1  if (idx >= 0 && idx < (int)active_comms.size()) {
2      // Sblocco dei successori condizionati dal
   completamento effettivo (requires)
3      unlock_requires(my_list, finished_task_id,
   my_ready_queue);
4
5      // Rilascio sicuro del buffer associato alla
   comunicazione conclusa
6      free(active_comms[idx].buffer);
7
8      // Strategia di rimozione O(1)
9      size_t last = active_comms.size() - 1;
10     if (idx != last) {
11         // Sovrascrittura con l'ultimo elemento
12         active_comms[idx] = active_comms[last];
13     }
14     // Rimozione fisica dell'elemento duplicato in coda
15     active_comms.pop_back();
16 }

```

Listing 3.4: Porzione di codice estratta da Schedulerpp.cpp durante la risoluzione di MPI_Waitany.

L'algoritmo esegue una sostituzione diretta sovrascrivendo l'oggetto deallocato in posizione `idx` con una copia dell'ultimo elemento valido del vettore (`last`), per poi accorciare la dimensione della struttura mediante una `pop_back()`. Questa scelta architetturale immunizza lo Scheduler da colli di bottiglia computazionali interni, garantendo la scalabilità di GRIT indipendentemente dalla densità dei messaggi asincroni iniettati nel sistema.

3.3.4 Esempio di simulazione

Si consideri un esempio di costruzione del grafo dal file:

```
1 rank 0 {  
2   l1: calc 120  
3   l2: isend 64b to 1 tag 100  
4   l3: calc 80  
5   l4: recv 64b from 2 tag 100  
6   l5: calc 40  
7   l2 requires l1  
8   l3 irequires l2  
9   l4 requires l1  
10  l5 requires l4  
11 }
```

In questo scenario, lo Scheduler di GRIT opera nel seguente modo:

1. **Parsing e Setup:** Durante la fase di caricamento, il modulo Parser costruisce il grafo e identifica le istruzioni originarie prive di vincoli antecedenti, assegnando loro un contatore di dipendenze pendenti pari a zero. Di conseguenza, l'istruzione l1 viene immediatamente inserita all'interno della Ready Queue per avviare l'esecuzione.
2. **Propagazione Bloccante:** Lo Scheduler estrae l1 ed emula il carico computazionale applicando la corretta sospensione temporale. Al termine la routine di sblocco si attiva per aggiornare il grafo, decrementando il contatore delle dipendenze dei successori logici. In questa circostanza, le istruzioni l2 e l4 diventano eleggibili per l'elaborazione e transano nella Ready Queue.
3. **Esecuzione Asincrona e Sovrapposizione:** L'istruzione l2 codifica un'istruzione non bloccante. Lo Scheduler invoca l'opportuna primitiva e sposta l'oggetto della richiesta (`MPI_Request`) nella Waiting Queue. La presenza di una dipendenza di tipo `irequires` fa sì che l'istruzione l3 venga sbloccata immediatamente.
4. **Sincronizzazione e Risoluzione delle Code:** L'istruzione l4 viene invece processata come operazione bloccante. L'emissione della direttiva `MPI_Recv` vincola temporaneamente l'avanzamento di quel segmento di grafo. Solamente al termine della ricezione completa del payload dal rank 2, il vincolo `requires` su l5 risulta soddisfatto, permettendo al blocco finale di calcolo di procedere. L'architettura dinamica dello Scheduler, combinando primitive `MPI_Test` per i controlli opportunistici e `MPI_Waitany` per le attese, garantisce che le operazioni

residenti nella Waiting Queue come l2 vengano completate e finalizzate anche durante i periodi di stallo imposti dalle comunicazioni bloccanti, annullando il rischio di deadlock silenti.

3.4 Meccanismi di simulazione

Il realismo di GRIT non deriva solo dalla corretta gestione logica delle dipendenze, ma anche dalla precisione con cui vengono emulate le fasi di occupazione della CPU e le risposte del sistema operativo.

3.4.1 Modellazione della computazione tramite Sospensione Temporale

Le istruzioni di tipo CALC rappresentano le fasi di calcolo dell'applicazione originale. In GRIT, queste non eseguono operazioni matematiche reali, ma simulano l'occupazione temporale del processore. L'implementazione nello Scheduler utilizza la libreria Thread, la quale sospende l'esecuzione del rank per la durata specificata nella traccia. Per garantire la massima fedeltà rispetto ai diversi standard di tracciamento, il software supporta quattro unità di misura temporali: nanosecondi, microsecondi, millisecondi e secondi.

Un elemento distintivo è l'introduzione dello scale factor. Questo parametro permette di dividere i tempi di calcolo per un fattore costante, consentendo di simulare scenari in cui la velocità di calcolo dei nodi (es. passaggio a GPU più performanti) aumenta, lasciando però invariate le prestazioni della rete. Tale meccanismo è fondamentale per identificare se un'applicazione sia limitata dalla computazione (computation-bound) o dalla comunicazione (communication-bound).

3.4.2 Iniezione del rumore di sistema: il modello Gaussiano

Per analizzare la robustezza di un'applicazione rispetto al rumore di sistema, GRIT integra un generatore di rumore basato sulla distribuzione normale Gaussiana, implementata nel Parser.

```
1 if (simulate_noise > 0.0) {  
2     // La deviazione standard e' proporzionale al tempo di  
3     calcolo originale  
4     double stddev = amount * simulate_noise;  
5     normal_distribution<double> dist(0.0, stddev);  
6     double noise = dist(*rng);  
7     amount = max(0.0, amount + noise);  
8 }
```

Listing 3.5: Codice per la generazione del rumore.

La logica di iniezione segue tre principi:

- **Proporzionalità:** La deviazione standard del rumore viene definita dall'utente come una frazione del tempo di calcolo originale. Se, ad esempio, si imposta un fattore di rumore dello 0.1, un'istruzione CALC da 1000 unità subirà una variazione basata su una distribuzione con media 0 e deviazione standard pari a 100.
- **Applicazione Pre-Scaling:** Il rumore viene sommato al valore originale prima dell'eventuale applicazione dello scale factor, garantendo che la varianza statistica rimanga coerente con il carico computazionale simulato.
- **Indipendenza tra Rank:** Ogni processo MPI utilizza un seme differente per il generatore di numeri casuali, replicando la natura asincrona e non correlata del rumore nei cluster reali.

L'inizializzazione del generatore di numeri pseudo-casuali avviene attraverso un seme che combina il tempo corrente di sistema con l'identificativo del rispettivo processo MPI. La componente temporale assicura la variabilità del rumore tra esecuzioni effettuate in momenti diversi. L'aggiunta del rank risolve invece il problema della simultaneità tipico dei sistemi HPC poiché tutti i task vengono avviati nello stesso istante, l'uso del solo tempo di sistema porterebbe ogni processo a generare una sequenza di rumore identica. Inoltre, per soddisfare le esigenze di verificabilità e debug, il framework permette di specificare un seme personalizzato direttamente da riga di comando. Questa opzione consente di ottenere esecuzioni fisse, deterministiche e riproducibili.

3.4.3 Supporto Multi-Formato e Flessibilità dello Scheduler

Lo Scheduler di GRIT è progettato per essere agnostico rispetto al formato di input. Attraverso un'analisi preliminare della traccia, il software rileva se le istruzioni seguono la semantica standard GOAL o quella GOALPP.

Questa flessibilità permette allo stesso motore di esecuzione di gestire sia il formato GOAL, dove il progresso asincrono deve essere forzato tramite un uso intensivo di `MPI_Test` per evitare stalli, sia il formato GOALPP.

Capitolo 4 - Valutazione Sperimentale e Analisi Prestazionale

4.1 Metodologia e ambiente di test

Per valutare sperimentalmente la fedeltà temporale e la scalabilità le sessioni di test e la profilazione del codice sono state eseguite avvalendosi del supercomputer Leonardo, ospitato in Italia presso il data center del Tecnopolo di Bologna e gestito dal consorzio interuniversitario CINECA [20]. Classificato costantemente ai vertici della graduatoria globale TOP500 [21], Leonardo rappresenta un'infrastruttura di classe pre-exascale sviluppata dall'impresa europea EuroHPC Joint Undertaking [22].

Il supercomputer Leonardo è equipaggiato con un'architettura di rete NVIDIA Mellanox HDR InfiniBand, la quale è in grado di garantire una larghezza di banda aggregata fino a 200 Gbps per nodo. L'impiego di una topologia di rete ad alto throughput ha permesso di sottoporre le code asincrone dello Scheduler interno a condizioni di latenza operative reali. Ciò assicura che la gestione delle comunicazioni non bloccanti e l'assorbimento dell'OS Jitter non subiscano distorsioni dovute a limiti di rete.

4.2 Il Validator

Allo scopo di comprovare la rigorosa aderenza della simulazione rispetto alle tempistiche e ai pattern di invio descritti dalla traccia originaria, si è progettato e implementato un modulo di validazione dedicato. All'interno di ambienti HPC distribuiti, è necessario dimostrare che un simulatore non alteri la semantica dell'esecuzione, in particolare quando si adottano motori asincroni basati su code di priorità. Questo componente, denominato Validator, garantisce che il flusso di istruzioni riproduca fedelmente il Grafo Aciclico Diretto prestabilito, prevenendo alterazioni dell'ordine causale delle comunicazioni o stalli artificiali.

4.2.1 Workflow di validazione

Nella prima fase del processo di validazione si avvia la simulazione della traccia in input tramite GRIT, abilitando contemporaneamente sia la generazione di log traccianti interni sia liballprof. Questa sovrapposizione consente al profiler di registrare ogni singola chiamata al layer MPI smistata dallo Scheduler.

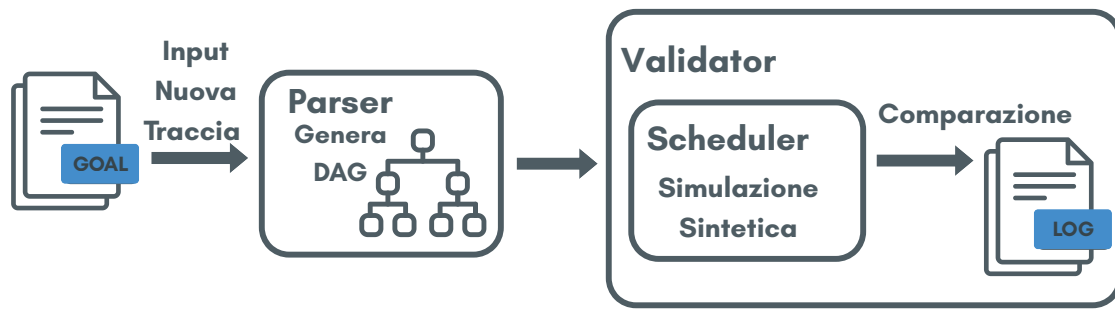


Figura 4.1: Flusso di esecuzione del modulo Validator.

A completamento dell'esecuzione, i log grezzi catturati dal profiler vengono processati dal tool Schedgen, il quale sintetizza una nuova traccia. Nella fase finale, GRIT viene riavviato attivando l'apposita modalità di verifica. In questo modo il software non si limita a un semplice confronto testuale statico bensì elabora la nuova traccia sintetizzata e ne ricostruisce integralmente il DAG.

L'aspetto fondamentale di questa fase risiede nell'emulazione dell'esecuzione, il Validator si sostituisce allo Scheduler standard e itera dinamicamente attraverso le code di dipendenza simulando l'avanzamento logico del programma generato dal DAG. A differenza di una simulazione ordinaria opera omettendo l'invocazione delle primitive MPI.

Si consideri il seguente frammento di log per il validator:

```

1 calc 120
2 send 64 1 100
3 recv 64 3 103
4 calc 80
5 allreduce 32 MPI_SUM
  
```

Listing 4.1: Frammento di log per il Validator.

La grammatica del log astrae la sintassi originale della traccia per esporre i soli parametri delle istruzioni. Le primitive punto-a-punto riportano sequenzialmente il payload in byte, il rank del partner di comunicazione e l'identificativo univoco del tag. Per le operazioni collettive nel formato GOALPP vengono tracciati i metadati specifici, quali ad esempio la dimensione del buffer e l'operatore di riduzione applicato nel caso della `allreduce`.

Questa standardizzazione priva la traccia di qualsiasi ambiguità interpretativa, consentendo al motore di validazione di operare un confronto logico e temporale deterministico. Man mano che le istruzioni vengono sbloccate ed eseguite sintetica-

mente, il modulo consuma riga per riga i log interni salvati durante la run originale. Il parser semantico confronta istantaneamente l'evento virtualizzato con lo storico reale. Il processo di validazione si considera superato esclusivamente qualora si riscontri un isomorfismo assoluto tra la sequenza logica appena eseguita e lo stream delle transazioni di rete originariamente tracciate.

4.2.2 Analisi dell'overhead e soglie di tolleranza nel Validator

L'intercettazione a livello di sistema delle chiamate di libreria comporta un degrado temporale fisiologico. Questo costo computazionale extra indotto dal profiler si manifesta all'interno della traccia ricompilata sotto forma di artefatti prestazionali: si palesano istruzioni computazionali con tempistiche artificialmente dilatate. Tale fenomeno è ampiamente noto come *probe effect*, ovvero la distorsione della cronologia degli eventi e del comportamento originale del sistema indotta dallo strumento di misurazione stesso.

Inoltre, per garantire il progresso asincrono della simulazione lo Scheduler gestisce le comunicazioni nel formato GOAL standard avvalendosi esclusivamente di primitive MPI non bloccanti. Durante la fase di validazione, il profiler intercetta queste chiamate asincrone registrandone i timestamp di inizio e fine. Tuttavia, in fase di sintesi della traccia, il tool Schedgen interpreta i micro-intervalli temporali necessari alla gestione di tali chiamate come attività di calcolo effettiva. Di conseguenza, un'operazione che originariamente era una singola istruzione non bloccante viene tradotta in una sequenza composta da una primitiva di comunicazione seguita da un'istruzione calc di minima entità.

Questa frammentazione introdotta artificialmente impedisce un confronto deterministico riga-per-riga tra il log interno di GRIT e la nuova traccia. Per ovviare a tale problema, si è implementata una funzione di Merge and Replace. Questo algoritmo analizza il log della nuova esecuzione e provvede a sommare ogni sequenza di istruzioni calc contigue, aggregandole in un unico blocco computazionale che precede o segue la comunicazione.

Scandendo iterativamente la coda delle dipendenze, il sistema somma in un registro il tempo di calcolo globale della nuova traccia e parallelamente quello originario. La differenza di tempo definisce l'Overhead totale. Quest'ultimo viene normalizzato rispetto al numero complessivo delle transazioni di rete per ottenere l'impatto medio per operazione. Il ciclo di validazione si dichiara superato esclusivamente se l'overhead medio per transazione rientra all'interno di una tolleranza massima.

4.3 Analisi Comparativa: Impatto dei formati GOAL vs GOALPP

Come discusso nei capitoli precedenti, la principale divergenza risiede nella gestione delle operazioni collettive, le quali vengono espanse in primitive punto-a-punto nel formato GOAL standard, mentre mantengono la loro semantica nativa nel formato GOALPP.

I test di ripetibilità condotti sulle quattro tracce hanno evidenziato come l'astrazione di alto livello del GOALPP influenzi positivamente le tempistiche di esecuzione, riducendo l'overhead di rete.

Come mostrato nel grafico in Figura 4.2 la traccia `openmx_32` presenta il divario più marcato: l'esecuzione in formato GOAL ha registrato un wall clock time massimo di circa 19,13 secondi, con un tempo di comunicazione che si attesta intorno ai 5,2 secondi. Eseguendo la medesima traccia in formato GOALPP, il tempo totale scende a circa 18,33 secondi, con tempo di comunicazione ridotto a circa 4,5 secondi. Tale flessione documenta come la preservazione della semantica collettiva consenta alla libreria MPI sottostante di applicare algoritmi ottimizzati.

Anche nelle tracce a minore intensità comunicativa, come `milc_32` e `lammps_32`, il formato GOALPP mostra un comportamento più stabile nei tempi di calcolo e comunicazione, sebbene i tempi totali, rispettivamente $\sim 5,40s$ e $\sim 7,42s$, rimangano marginalmente simili tra le due varianti.

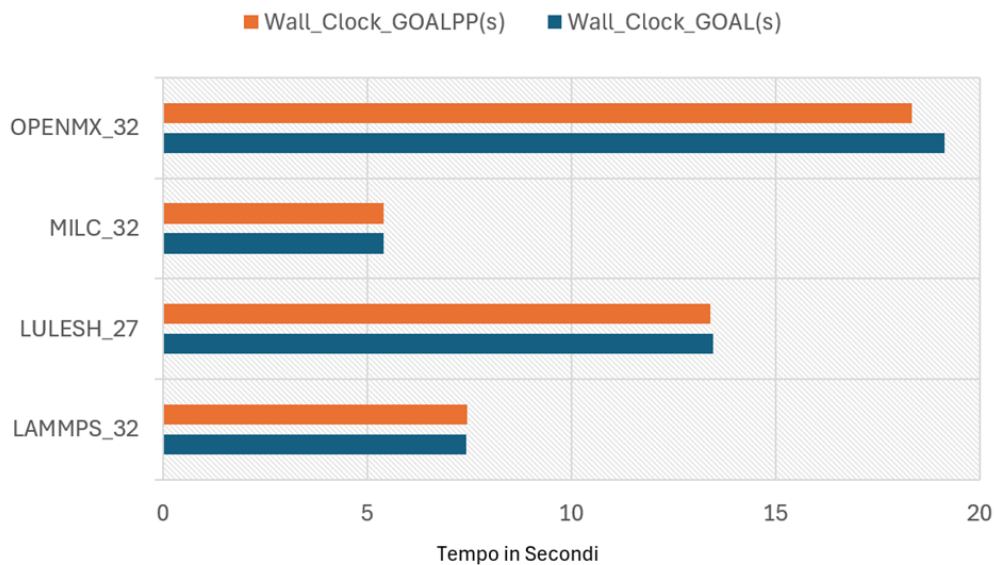


Figura 4.2: Grafico comparativo dei tempi di esecuzione per le tracce in formato GOAL vs GOALPP.

4.4 Studio di Scaling della Computazione

Nell'ambito della simulazione trace-driven, la manipolazione dei tempi di calcolo mediante scaling permette di modellare le prestazioni dei nodi di calcolo in molteplici scenari hardware, facilitando l'isolamento del comportamento del sottosistema di rete. L'integrazione di un moltiplicatore, definito scale factor, abilita la conduzione di analisi predittive su diverse architetture di calcolo, consentendo di stimare l'impatto prestazionale ancor prima della sua effettiva disponibilità fisica.

Sottoponendo le tracce a incrementi progressivi del fattore di scaling (da 1 fino a 250), si osserva un decadimento asintotico del wall-clock time totale. Attraverso la divisione dei tempi associati alle istruzioni di calcolo (CALC), l'incidenza della computazione viene abbattuta quasi totalmente nelle configurazioni estreme, facendo emergere in modo netto la natura rigida del sottosistema di comunicazione.

Analizzando i dati relativi alla traccia `lulesh_27` (formato GOAL) nel grafico in Figura 4.3, il tempo totale di esecuzione decresce da 13,44 secondi (Scale Factor 1) fino ad attestarsi su un valore di circa 8,17 secondi (Scale Factor 250). Si rileva che, per fattori di scaling superiori a 100, la curva dei tempi tende all'appiattimento: il limite prestazionale cessa di essere determinato dalla potenza della CPU e diventa dipendente dall'infrastruttura di interconnessione, la quale mantiene un tempo di comunicazione incompressibile di circa 8,15 secondi.

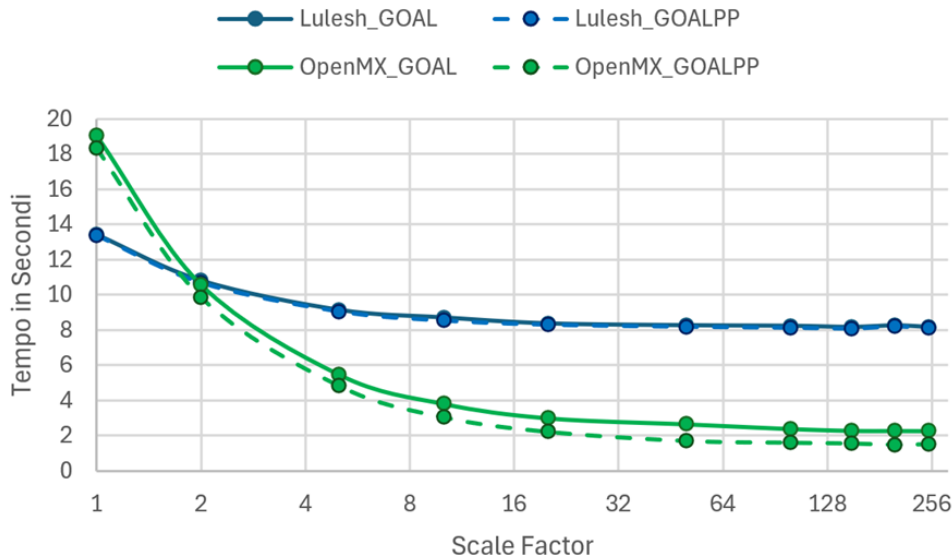


Figura 4.3: Grafico comparativo dei tempi di esecuzione soggetti a Scale Factor per le tracce in formato GOAL vs GOALPP.

Un comportamento analogo, seppur con una dinamica più marcata, è riscontrabile nella traccia `openmx_32` (formato GOAL). In questo caso, l'applicazione transita

da un tempo iniziale di 19,08 secondi a soli 2,27 secondi con uno Scale Factor di 250. Il punto di flesso si manifesta già intorno a fattori di scaling pari a 50, oltre i quali il wall clock time rimane ancorato a un valore di circa 2,2 secondi, dettato interamente dall'overhead di rete. Tali riscontri confermano l'efficacia del parametro `-scale-factor` nell'isolare le componenti temporali della simulazione, permettendo di identificare con precisione la soglia di saturazione oltre la quale l'investimento in risorse computazionali non produce ulteriori benefici prestazionali.

4.4.1 Identificazione dei colli di bottiglia: Compute-bound vs Network-bound

La scomposizione del wall-clock time nelle componenti di calcolo (calculation time) e comunicazione (communication time) ha permesso di classificare i carichi di lavoro simulati tramite il tool GRIT, evidenziando i limiti architetturali dominanti che ne condizionano la scalabilità, identificando i colli di bottiglia architetturali.

Si consideri il caso della traccia `lulesh_27`, la quale manifesta un comportamento intrinsecamente network-bound. Durante i test condotti nel formato GOAL con Scale Factor unitario, il tempo assorbito dalle operazioni di comunicazione (8,39 s) risulta già predominante rispetto a quello computazionale (5,04 s). Estremizzando lo scenario tramite un incremento dello scaling a un valore pari a 250, l'impatto del calcolo viene quasi del tutto annullato (0,02 s); tuttavia, il wall-clock time complessivo si stabilizza asintoticamente attorno al valore di 8,17 s, rivelando come il costo temporale sia governato interamente dall'overhead comunicativo.

In scenari operativi affini a LULESH, ottimizzare unicamente il calcolo risulta inefficace senza un adeguato potenziamento del sottosistema di rete. Tale limite prestazionale, tuttavia, è raramente imputabile a un deficit della banda di bisezione globale, derivando piuttosto dai vincoli di injection bandwidth del singolo nodo computazionale. Nello specifico, la semantica del formato GOAL espande le primitive collettive in dense sequenze di istruzioni punto-a-punto asincrone, questa mole di piccoli messaggi satura rapidamente le capacità di serializzazione del controller di rete locale, generando ritardi a cascata sui buffer interni ben prima che il traffico raggiunga l'infrastruttura di rete globale.

Di contro, `openmx_32` si presenta inizialmente come un'applicazione meno communication-bound, con calcolo $\sim 14,7$ s e comunicazione $\sim 4,3$ s. Tuttavia, l'analisi rivela un punto di flesso: superato lo Scale Factor 20, dove calcolo e comunicazione si equivalgono a circa 2,3s nel formato GOAL, l'applicazione subisce una mutazione di stato, diventando anch'essa limitata dalla rete.

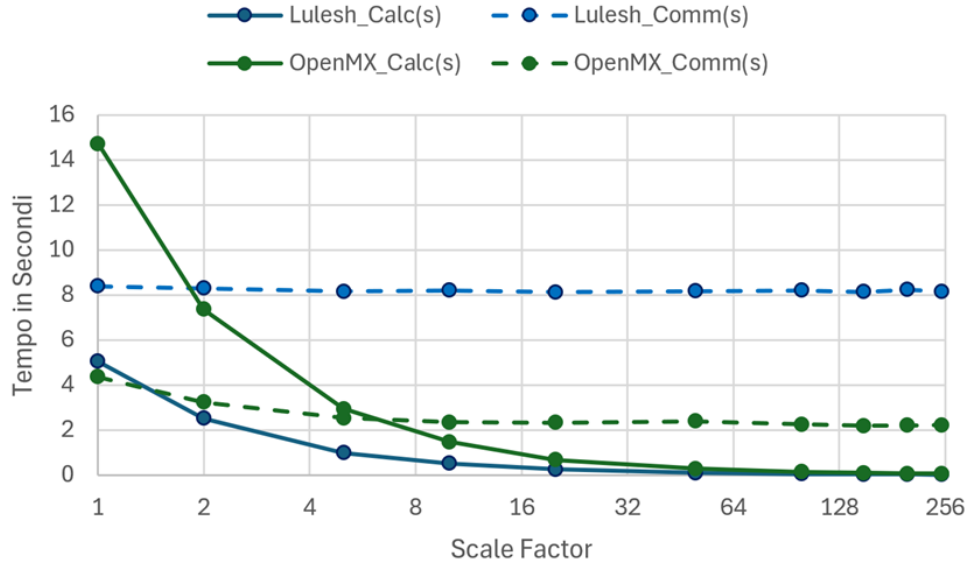


Figura 4.4: Grafico comparativo dei tempi di esecuzione soggetti a Scale Factor per le tracce in formato GOAL.

Infine è possibile osservare in Figura 4.4 un calo controintuitivo dei tempi di comunicazione (da 4,3s a 3,3s) da parte di OpenMX, sebbene si stia riducendo esclusivamente il carico computazionale, nelle tracce GOAL, le istruzioni di calcolo sono densamente alternate alle primitive di rete. Se un processo invoca una `MPI_Recv` mentre il mittente è ancora impegnato in un blocco di computazione antecedente alla relativa `MPI_Send`, il ricevitore subirà un tempo di inattività in attesa del dato. Di conseguenza, diminuendo lo scale factor, si contrae la durata delle istruzioni `calc`, abbattendo di riflesso il tempo di attesa passiva. La riduzione dei tempi di comunicazione non rappresenta quindi un artefatto, bensì la mitigazione degli idle time.

Questo fenomeno si ricollega direttamente alle peculiarità architetturali di OpenMX discusse nel Capitolo 1. Poiché il software genera un traffico globale e sincrono di tipo all-to-all, la fluidità dell'intera infrastruttura di rete è rigidamente vincolata alla puntualità dei singoli nodi mittenti. Diminuendo lo scale factor, GRIT accelera la componente computazionale, eliminando le barriere di desincronizzazione locali. La contrazione del tempo di comunicazione dimostra in modo diretto che le prestazioni sono limitate dai colli di bottiglia della rete, una volta rimosso il disallineamento temporale causato dalla CPU emerge il profilo di comunicazione puro dell'applicazione.

4.5 Impatto del Rumore sulla Sincronizzazione Globale

Nei cluster HPC di produzione, le prestazioni complessive sono frequentemente inficiate da perturbazioni temporali non deterministiche a livello di sistema operativo, note in letteratura come OS Jitter. Al fine di quantificare la resilienza delle applicazioni a tali anomalie, il framework GRIT integra un meccanismo di iniezione di rumore stocastico, modellato mediante una distribuzione gaussiana applicata dinamicamente ai tempi di calcolo, ovvero le istruzioni `CALC`. L'utilizzo di questo simulatore di disturbo è cruciale per comprendere come minime deviazioni locali possano riverberarsi sulle performance globali del sistema.

L'analisi empirica condotta sulle tracce dimostra formalmente come variazioni microscopiche a livello di singolo nodo si propaghino in modo macroscopico sull'infrastruttura di comunicazione, innescando un netto degrado prestazionale sistemico. Esaminando i risultati relativi alla traccia `openmx_32` processata nel formato GOAL standard, si rileva che un'esecuzione in assenza di rumore (livello 0.0) garantisce un wall clock time totale pari a 19,11 secondi, dei quali 5,03 secondi sono attribuibili all'overhead di rete. Introducendo un fattore di perturbazione elevato (livello 0.7), il tempo complessivo di esecuzione subisce una severa dilatazione, raggiungendo i 33,90 secondi (Figura 4.5).

Il dato di maggiore rilevanza tecnica risiede nella scomposizione di tale scostamento: l'incremento non è imputabile al mero costo computazionale aggiuntivo, bensì a un'esplosione anomala del communication time, che transita drasticamente da 5,03 a 23,83 secondi. Tale riscontro fornisce un'evidenza empirica diretta del cosiddetto "effetto valanga" intrinseco negli ambienti di calcolo distribuiti. Nel momento in cui un rank subisce un rallentamento locale causato dal jitter e giunge in ritardo a una barriera di sincronizzazione o a un'operazione collettiva bloccante, i processi dipendenti (peer) sono costretti ad accumulare cicli di inattività (idle time). Questo intervallo di attesa forzata viene inevitabilmente registrato dalle primitive MPI come overhead comunicativo, falsando la percezione dell'efficienza reale della topologia di rete.

Analoghe criticità si palesano dall'analisi di tracce caratterizzate da una minore intensità assoluta di comunicazione, come `milc_32`. In questo scenario, l'esposizione a un rumore di magnitudo 0.7 quasi decuplica i tempi associati alla rete, spingendoli da 0,91 a 8,10 secondi. Questi risultati validano sperimentalmente l'assunto teorico discusso nello studio di Petrini, Kerbyson e Pakin [3], confermando che la stabilità prestazionale del sistema operativo e la conseguente minimizzazione dell'OS Jitter costituiscano dei prerequisiti architetturali imprescindibili per garantire la scalabilità efficiente delle applicazioni HPC su scala Exascale.

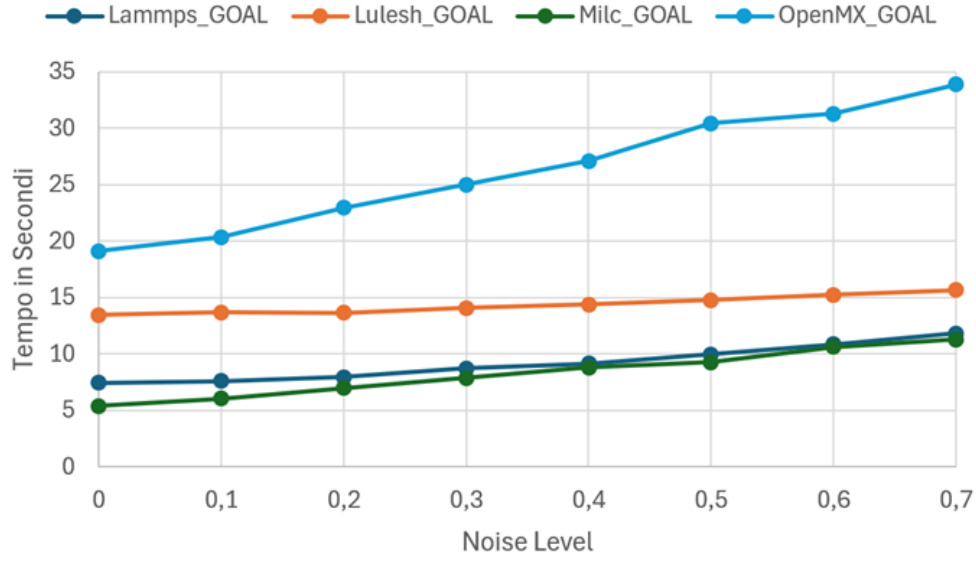


Figura 4.5: Grafico comparativo dei tempi di esecuzione soggetti a noise per le tracce in formato GOAL.

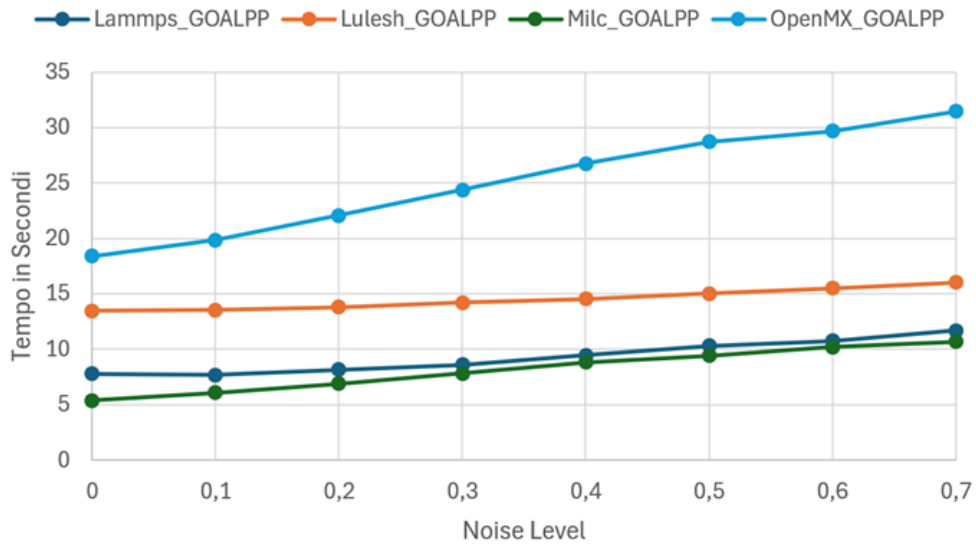


Figura 4.6: Grafico comparativo dei tempi di esecuzione soggetti a noise per le tracce in formato GOALPP.

4.6 Analisi Prestazionale

L'obiettivo primario è quantificare il costo di inizializzazione distribuita e l'impatto sulle risorse di memoria all'aumentare del grado di parallelismo. I benchmark sono stati eseguiti adottando le tracce della miniapp LULESH, scalate su configurazioni a 8, 27 e 64 rank. Tutte le tracce analizzate in questa fase sono espresse in formato GOAL standard. Per garantire l'isolamento hardware e l'affidabilità delle metriche,

l'esecuzione è stata demandata al workload manager SLURM, estrapolando i consumi di memoria tramite l'utilità di sistema `sacct`.

4.6.1 Analisi dell'Efficienza del Modulo Parser

Il primo parametro di valutazione riguarda l'efficienza della fase di caricamento della traccia. Avendo delegato il parsing in modo del tutto indipendente a ciascun rank, si intende dimostrare che l'I/O concorrente sul file system parallelo non costituisca un collo di bottiglia critico all'aumentare dei processi.

Dall'estrazione dei log operativi, si evince la seguente progressione del carico di inizializzazione:

- **8 Rank:** Il modulo elabora mediamente 95.000 istruzioni per rank, per un volume aggregato stimabile in circa 760.000 istruzioni. Il tempo di completamento del parsing si attesta su una media di 0.56 secondi.
- **27 Rank:** Il numero di istruzioni processate dal singolo rank subisce un incremento marcato, raggiungendo picchi di 288.162 operazioni. La lettura viene completata in 2,62 secondi.
- **64 Rank:** Il volume massimo di istruzioni per rank raggiunge le 290.163 istruzioni. Il tempo di inizializzazione registra una media di 7,35 secondi.

I dati raccolti evidenziano che il tempo di parsing manifesta una progressione lineare direttamente proporzionale alla dimensione fisica del file di traccia. Da un lato, questa metrica convalida l'efficacia dell'algoritmo a scansione mirata descritto nel Capitolo 3. L'eliminazione delle barriere di sincronizzazione inter-processo in fase di avvio e la pre-allocazione dello spazio di memoria garantiscono tempi di caricamento sostenibili anche in presenza di un volume elevato di istruzioni. Dall'altro lato, sebbene i meccanismi implementati in GRIT contengano l'impatto computazionale di questa fase, il persistere di tale andamento lineare delinea un limite strutturale intrinseco al formato GOAL stesso.

A causa dell'organizzazione testuale del formato, ciascun Rank MPI è costretto a scorrere sequenzialmente il file per localizzare ed estrarre la porzione di codice e le dipendenze di propria competenza. Di conseguenza, all'aumentare della scala del sistema, la dimensione complessiva del file subisce un'espansione proporzionale alla quantità di processi simulati. Questo fenomeno amplifica l'overhead di scansione testuale e di scomposizione delle stringhe a carico di ogni singolo core, trasformando la fase di inizializzazione in un collo di bottiglia I/O legato alla natura stessa del formato di input, indipendentemente dall'efficienza del parser sviluppato.

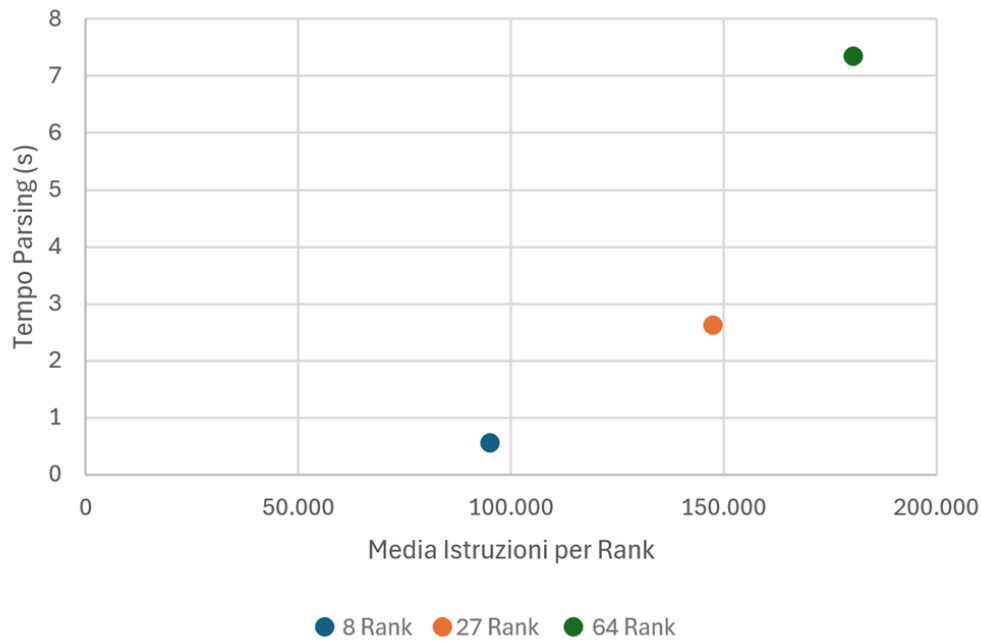


Figura 4.7: Grafico comparativo del numero di istruzioni processate per rank durante la fase di parsing.

4.6.2 Profiling della Memoria e Allocazione Dinamica

In ambito HPC, il consumo di RAM, Memory Footprint, rappresenta un limite stringente per la scalabilità dei simulatori. L'obiettivo in questa fase è verificare l'efficacia della strategia di allocazione dinamica dei payload estesi, progettata per mantenere compatta la struttura dati del Grafo Aciclico Diretto.

La misurazione è stata condotta monitorando il parametro Maximum Resident Set Size (MaxRSS) registrato dal sistema batch al termine di ciascun job:

- **Configurazione a 8 Rank:** 762.670 istruzioni complessive, determinando un carico medio di 95.334 istruzioni per rank. La distribuzione si mantiene omogenea tra i processi, oscillando da un valore minimo di 88.312 istruzioni fino a un valore massimo di 102.356 istruzioni. In questo scenario, il picco di memoria registrato per nodo si attesta su 382.040 KB.
- **Configurazione a 27 Rank:** Il numero totale di istruzioni aggregate raggiunge quota 3.984.283, con un'allocazione media di 147.566 istruzioni per rank. A causa delle interfacce geometriche interne, emerge un'asimmetria nel bilanciamento del grafo locale, che varia da un minimo di 92.048 istruzioni fino a un picco massimo di 288.162 operazioni a carico del rank 13. Nonostante l'incremento del volume di istruzioni, l'occupazione di picco della RAM per nodo evidenzia una stabilità notevole, assestandosi a 413.456 KB.

- **Configurazione a 64 Rank:** Con il massimo grado di parallelismo testato, si ha una dimensione del DAG globale pari a 11.546.654 istruzioni complessive, corrispondenti a una media di 180.416 istruzioni per rank. L'eterogeneità del carico si estende da un minimo di 94.048 istruzioni a un valore massimo di 290.163 istruzioni. Sotto questo scenario di stress informativo, l'allocazione massima di memoria catturata a livello di sistema subisce una flessione, registrando un valore di picco pari a 235.704 KB per nodo.

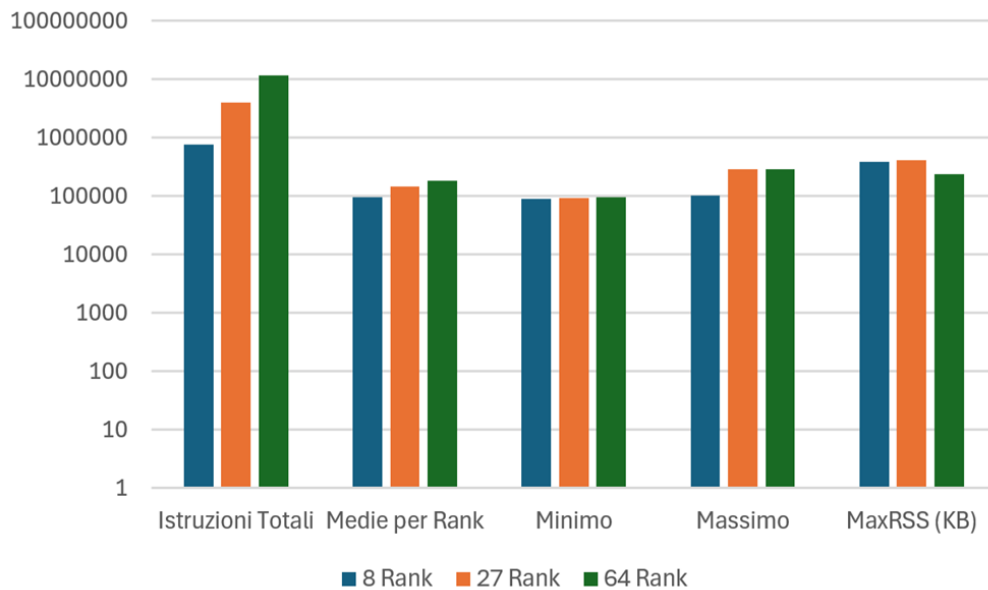


Figura 4.8: Grafico comparativo delle metriche di carico computazionale (istruzioni totali, medie, minime e massime per rank) e dell'occupazione di memoria massima (MaxRSS espressa in KB) al variare del parallelismo del sistema (8, 27 e 64 Rank). Il grafico evidenzia la notevole stabilità del picco di memoria allocata per nodo e l'incremento dell'asimmetria di carico (*load imbalance*) nelle configurazioni a più alto numero di processi.

Per interpretare l'andamento dei consumi hardware è necessario scomporre l'impronta di memoria nei suoi tre contributi principali: l'overhead fisso del runtime di sistema, il dimensionamento statico del vettore delle istruzioni e l'allocazione dinamica sull'heap dei metadati.

Il primo fattore determinante risiede nella stretta correlazione tra la topologia di scomposizione del dominio di LULESH e il volume di istruzioni locale assegnato a ciascun processo parallelo. Nella configurazione a 8 rank, il carico appare omogeneo e limitato, con un picco massimo di 102.356 istruzioni sul rank 0. Al contrario, nel passaggio a 27 rank, la complessità determina un'asimmetria nel carico di tracciamento: il rank 13 si trova a dover gestire un grafo locale massiccio da 288.162 istruzioni.

Questo incremento del 181% nel numero di nodi del DAG locale si riflette in modo estremamente contenuto sul consumo di RAM, che fa registrare un aumento di appena l'8,2% passando da circa 382 MB a 413 MB. Tale andamento fornisce la prova empirica della validità delle scelte di ottimizzazione implementate nel codice. Considerando che la struttura base `Instruction` presenta una dimensione fissa inferiore ai 100 byte, il vettore principale per il rank più gravato occupa una porzione di memoria lineare e prevedibile, quantificabile in circa:

$$\text{Memoria_vettore} = 288.162 \times 100 \text{ byte} \approx 28,8 \text{ MB}$$

La quota restante del `MaxRSS` osservato è saturata dall'overhead intrinseco dell'ambiente di esecuzione MPI e dalle librerie di sistema caricate a runtime.

Il comportamento rilevato nella configurazione a 64 rank, dove il consumo massimo per nodo si flette a 235.704 KB, introduce una considerazione fondamentale sulla granularità della rendicontazione dei job in ambiente supercomputazionale. Sebbene il picco massimo di istruzioni per singolo processo rimanga analogo al caso precedente (290.163 istruzioni sul rank 21), l'allocazione del job su un numero maggiore di nodi computazionali distinti rimodula la densità dei processi per singolo socket hardware. Di conseguenza, i pool di memoria interni condivisi e i buffer di comunicazione associati ai canali virtuali dell'infrastruttura Open MPI beneficiano di una netta riduzione della contesa delle risorse a livello locale.

Dal punto di vista della teoria dei sistemi distribuiti, l'analisi di questo trend dimostra che il framework GRIT raggiunge una scalabilità dello spazio di memoria ottimale. L'impronta di memoria del simulatore non è legata in modo quadratico o lineare al numero globale di rank del cluster, bensì è strettamente confinata dal grado di parallelismo locale e dalla dimensione del sotto-grafo di competenza del singolo rank.

Capitolo 5 - Conclusioni e Sviluppi Futuri

5.1 Sintesi dei risultati ottenuti

L'avvento di architetture HPC su scala pre-exascale ed exascale ha reso necessaria l'adozione di metodologie di simulazione predittiva per la valutazione delle prestazioni del sottosistema di rete, isolandolo dalla pura potenza computazionale dei nodi. In risposta a tale esigenza, il presente lavoro di tesi ha illustrato la progettazione, l'implementazione e la validazione del framework GRIT, GOAL tRaces Injection Tool, una Proxy Application distribuita in grado di riprodurre dinamicamente il traffico MPI estrapolato da applicazioni reali.

L'analisi sperimentale ha corroborato la robustezza architetturale del sistema. La fase di validazione condotta orchestrando il profiler liballprof e lo strumento Schedgen appartenenti all'ecosistema LogGOPSim [14] ha certificato la fedeltà causale e temporale del motore di scheduling interno. Il controllo rigoroso dell'ordine di esecuzione, con un overhead medio per transazione mantenuto saldamente al di sotto delle soglie critiche di tolleranza, ha dimostrato che GRIT non introduce artefatti architetturali in grado di inficiare i test prestazionali.

Infine, lo studio sulle perturbazioni ambientali, realizzato tramite l'iniezione di rumore gaussiano nelle fasi di calcolo, ha permesso di quantificare empiricamente il fenomeno dell'OS Jitter. I risultati hanno confermato sperimentalmente che variazioni locali infinitesime si propagano in modo catastrofico lungo il grafo delle dipendenze, innescando un "effetto valanga" che satura i tempi di attesa nelle sincronizzazioni globali.

5.2 Considerazioni sulla scalabilità del tool GRIT

La scalabilità rappresenta un requisito fondamentale per l'impiego di uno strumento di iniezione di tracce su grandi infrastrutture. GRIT adotta un'architettura completamente distribuita, i test condotti sul supercomputer Leonardo confermano la validità di questo approccio sia nella fase di inizializzazione che in quella di esecuzione.

La fase di caricamento dei dati, affidata al Modulo Parser, viene eseguita localmente e in parallelo da ogni rank. I risultati sperimentali mostrano che il tempo di caricamento scala in modo lineare con la dimensione del file di traccia: si passa da una media di 0,56 secondi per 8 rank a 7,35 secondi per 64 rank. Anche quando il grafo globale delle dipendenze supera gli 11.5 milioni di istruzioni totali, l'assenza di un punto di sincronizzazione centrale permette di eliminare i colli di bottiglia nel

setup, legando il tempo di avvio quasi esclusivamente alle prestazioni di lettura del file system parallelo.

Per quanto riguarda l'occupazione di memoria, l'uso dei puntatori intelligenti garantisce un'ottima scalabilità spaziale. L'allocazione dinamica dei payload complessi solo quando strettamente necessario permette di mantenere l'utilizzo della RAM al di sotto dei 414 MB per nodo, con una riduzione a circa 235 MB nella configurazione a 64 rank dovuta alla distribuzione dei processi su più nodi fisici. Questi dati escludono il rischio di anomalie da esaurimento di memoria, dimostrando la sostenibilità del tool anche a fronte di un numero elevato di istruzioni medie per rank, fino a oltre 180.000 operazioni locali a 64 processi.

I test di prestazione eseguiti in regime di weak scaling sulle tracce dell'applicazione benchmark LULESH confermano la capacità dello Scheduler Unificato di assecondare l'incremento del numero di processi senza indurre stalli architetturali. Coerentemente con la legge di Gustafson, a fronte di un tempo di calcolo puro per singolo core che si mantiene stabile a circa 5 secondi, il wall-clock time complessivo risente del fisiologico aumento dell'overhead di comunicazione introdotto dalla rete.

Nello specifico di questo scenario sperimentale, si rileva una netta transizione del profilo dell'applicazione: si passa da un comportamento prevalentemente compute-bound nella configurazione a 8 rank, in cui l'attività computazionale assorbe il 63% del tempo di esecuzione totale, a un profilo asintoticamente network-bound nella configurazione a 64 rank, dove le funzioni di comunicazione giungono a occupare il 69% del tempo complessivo.

L'uso combinato di routine asincrone non bloccanti permette di gestire questa crescita del traffico di rete in modo efficiente. GRIT si dimostra così in grado di riprodurre fedelmente il comportamento asincrono della rete reale e l'impatto delle asimmetrie di carico della decomposizione del dominio, senza generare stalli interni o deadlock nello scheduler.

5.3 Sviluppi Futuri: Integrazione dei Carichi AI e il Formato GOAL NCCL

La linea di sviluppo futuro di GRIT prevede l'estensione del framework alla simulazione predittiva dei carichi di lavoro associati all'addestramento distribuito di Large Language Models (LLM). A differenza degli applicativi HPC tradizionali basati sullo standard MPI, i cluster dedicati al deep learning impiegano librerie di comunicazione collettiva ottimizzate per interconnessioni, tra le quali riveste un ruolo predominante la NVIDIA Collective Communication Library (NCCL) [23]. Per

astrarre la complessità di tali interazioni su vasta scala, la toolchain ATLAHS [24] consente di intercettare l'esecuzione dei framework AI e di generare un grafo aciclico diretto Directed Acyclic Graph (DAG) compatibile con l'ecosistema di simulazione di LogGOPSIm. L'integrazione di questo standard di tracciamento, denominato GOAL NCCL, costituisce l'obiettivo dei prossimi sviluppi architetturali di GRIT.

5.3.1 Semantica e Mappatura delle Risorse nel Formato NCCL

Il formato GOAL NCCL espande la semantica dei metadati associati alle istruzioni per adattarsi all'esecuzione asincrona e multi-stream degli acceleratori grafici. All'interno di questo modello, ciascun rank MPI corrisponde univocamente a un singolo nodo fisico del cluster distribuito. La ripartizione interna delle risorse segue una gerarchia specifica:

- **Identificatore `cpu`:** Astrae un compute stream (o channel) logico allocato da NCCL. Istruzioni contrassegnate con il medesimo valore di `cpu` appartengono alla stessa lane logica del nodo e vengono serializzate. `cpu` distinte rappresentano flussi indipendenti dotati di parallelismo implicito, in grado di avanzare simultaneamente in assenza di dipendenze causali dirette.
- **Identificatore `nic`:** Identifica la GPU locale o l'interfaccia di rete associata, agendo come punto di serializzazione hardware per le operazioni che impegnano il mezzo trasmissivo.

È importante notare che la scomposizione dei rami paralleli non deve essere forzata tramite l'inserimento manuale di dipendenze rigide (requires) nel DAG prima della simulazione e l'aggiunta di vincoli fittizi alterano la topologia originaria della traccia e sono la causa primaria di stalli logici e deadlock in fase di esecuzione.

Pertanto l'avanzamento asincrono deve essere strutturato affinché un'operazione blocchi esclusivamente la porzione di grafo gerarchicamente dipendente da essa all'interno del proprio stream, senza interrompere l'avanzamento dell'intero rank.

Qualora istruzioni parallele eseguite su `cpu` differenti richiedano simultaneamente l'accesso alla stessa `nic`, la contesa sulla risorsa fisica condivisa viene gestita direttamente dalle code di accodamento dello Scheduler a runtime. Questo approccio emula l'arbitraggio hardware reale descritto nel modello LogGOPSIm, evitando la manipolazione statica del grafo.

5.3.2 Analisi del Tracciato ed Estensione dei Moduli di GRIT

Il seguente estratto mostra la sintassi del formato GOAL NCCL:

```

1 rank 0 {
2     l1: calc 300 cpu 1
3     l2: calc 150 cpu 2
4     l3: calc 500 cpu 3
5     l4: send 65536b to 1 tag 7001 cpu 1 nic 0
6     l4 requires l1
7     l5: send 131072b to 2 tag 7002 cpu 2 nic 0
8     l5 requires l2
9     l6: recv 65536b from 3 tag 8001 cpu 3 nic 0
10    l6 requires l3
11    l7: calc 200 cpu 1
12    l7 requires l4
13 }
```

Listing 5.1: Estratto di traccia GOAL standard

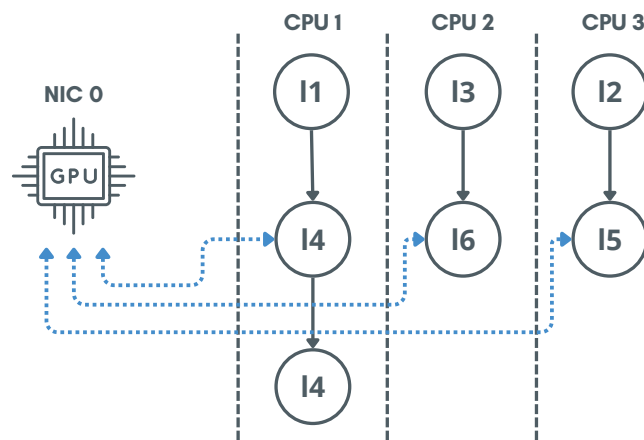


Figura 5.1: Rappresentazione del Grafo Aciclico Diretto (DAG) relativo all'estratto di traccia GOAL NCCL. Il diagramma evidenzia il parallelismo degli stream (cpu 1, cpu 2, cpu 3) e la comunicazione verso l'interfaccia di rete fisica condivisa (nic 0) del nodo, in questo caso il Rank 0.

L'analisi della traccia evidenzia le seguenti dinamiche operative:

1. **Esecuzione Parallela:** Le istruzioni l1, l2 e l3 operano su stream indipendenti (cpu 1, cpu 2, cpu 3). Lo Scheduler le avvia in parallelo poiché prive di vincoli `requires` reciproci.

2. **Risoluzione dei Vincoli:** Le operazioni di rete 14, 15 e 16 vengono abilitate non appena terminano i rispettivi calcoli locali sui canali dedicati.
3. **Gestione della Contesa:** Le primitive 14, 15 e 16 richiedono contemporaneamente l'accesso a `nic 0`. Lo Scheduler mantiene i task pronti in parallelo e ne indicizza l'accesso sul mezzo fisico secondo le metriche di latenza del modello LogGP. L'istruzione di calcolo successiva 17 (su `cpu 1`) viene sbloccata non appena si esaurisce l'impegno di 14 sulla rete, indipendentemente dallo stato di avanzamento dei flussi concorrenti su `cpu 2` e `cpu 3`.

Il supporto nativo di questa logica richiede il refactoring dei seguenti componenti:

- **Parser:** La struttura dati `Instruction` deve includere campi interi per memorizzare i token `cpu` e `nic`. La grammatica del modulo viene estesa per estrarre e validare tali metadati in fase di lettura della traccia.
- **Scheduler:** La Ready Queue globale per singolo processo viene sostituita da un array di code prioritarie aggregate per canale logico (`cpu`), asservite a un vettore di stato per il monitoraggio transitorio delle contese sulle `nic`.

Questo approccio architetturale consente a GRIT di operare non solo nell'ambito MPI ma anche nelle infrastrutture HPC dedicate all'addestramento dell'intelligenza artificiale.

Bibliografia

- [1] Jack Dongarra et al. «The International Exascale Software Project Roadmap 1». In: *IJHPCA* 25 (feb. 2011), pp. 3–60. DOI: 10.1177/1094342010391989.
- [2] Ahmad Maroof Karimi et al. «Exploring the Frontiers of Energy Efficiency using Power Management at System Scale». In: *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2024, pp. 1835–1844. DOI: 10.1109/SCW63240.2024.00230.
- [3] Fabrizio Petrini, Darren J. Kerbyson e Scott Pakin. «The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q». In: *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*. SC '03. Phoenix, AZ, USA: Association for Computing Machinery, 2003, p. 55. ISBN: 1581136951. DOI: 10.1145/1048935.1050204. URL: <https://doi.org/10.1145/1048935.1050204>.
- [4] Robert Preissl et al. «Detection of Collective MPI Operation Patterns». In: *Euro-Par 2008 – Parallel Processing*. Vol. 5168. Lecture Notes in Computer Science. Springer, 2008, pp. 119–128. DOI: 10.1007/978-3-540-85451-7_13.
- [5] Ian Karlin et al. *LULESH Programming Model and Performance Ports Overview*. Rapp. tecn. Lawrence Livermore National Laboratory (LLNL), Livermore, CA (United States), dic. 2012. DOI: 10.2172/1059462. URL: <https://www.osti.gov/biblio/1059462>.
- [6] Ian Karlin, Jeff Keasler e J Robert Neely. *LULESH Programming Model and Optimization Report*. Rapp. tecn. LLNL-TR-608821. Lawrence Livermore National Laboratory (LLNL), mag. 2013. DOI: 10.2172/1132577. URL: <https://osti.gov>.
- [7] Taisuke Ozaki et al. *OpenMX: Open source package for Material eXplorer*. <http://www.openmx-square.org/>. 2003.
- [8] Taisuke Ozaki. «Variationally optimized atomic orbitals for large-scale electronic structures». In: *Physical Review B* 67.15 (2003), p. 155108. DOI: 10.1103/PhysRevB.67.155108.
- [9] Marc Orio, Dimitrios A Pantazis e Frank Neese. «Density functional theory». In: *Photosynthesis Research* 102.2-3 (2009), pp. 443–453. DOI: 10.1007/s11120-009-9404-8.
- [10] James W. Cooley e John W. Tukey. «An Algorithm for the Machine Calculation of Complex Fourier Series». In: *Mathematics of Computation* 19.90 (1965), pp. 297–301. DOI: 10.2307/2003913. URL: <https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/Cooley65.pdf>.

- [11] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 3.0*. 2012. URL: <https://www.mpi-forum.org/docs/mpi-3.0/mpi30-report.pdf>.
- [12] William Gropp, Ewing Lusk e Anthony Skjellum. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. 3rd. Cambridge, MA, USA: MIT Press, 2014. ISBN: 978-0262527392.
- [13] Rajeev Thakur, Rolf Rabenseifner e William Gropp. «Optimization of Collective Communication Operations in MPICH». In: *International Journal of High Performance Computing Applications* 19.1 (2005), pp. 49–66.
- [14] Torsten Hoefler, Timo Schneider e Andrew Lumsdaine. «LogGOPSim: simulating large-scale applications in the LogGOPS model». In: *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*. HPDC '10. Chicago, Illinois: Association for Computing Machinery, 2010, pp. 597–604. ISBN: 9781605589428. DOI: 10.1145/1851476.1851564. URL: <https://doi.org/10.1145/1851476.1851564>.
- [15] David Culler et al. «LogP: towards a realistic model of parallel computation». In: *SIGPLAN Not.* 28.7 (lug. 1993), pp. 1–12. ISSN: 0362-1340. DOI: 10.1145/173284.155333. URL: <https://doi.org/10.1145/173284.155333>.
- [16] Albert Alexandrov et al. «LogGP: Incorporating Long Messages into the LogP Model for Parallel Computation». In: *Journal of Parallel and Distributed Computing* 44.1 (1997), pp. 71–79. ISSN: 0743-7315. DOI: <https://doi.org/10.1006/jpdc.1997.1346>. URL: <https://www.sciencedirect.com/science/article/pii/S0743731597913460>.
- [17] Martin Schulz. «Extracting critical path graphs from MPI applications». In: *2005 IEEE International Conference on Cluster Computing*. IEEE, 2005, pp. 1–10. DOI: 10.1109/CLUSTER.2005.347065.
- [18] Jørgen Bang-Jensen e Gregory Z. Gutin. *Digraphs: Theory, Algorithms and Applications*. 2nd. London: Springer-Verlag, 2008. ISBN: 978-1-84800-997-4.
- [19] Bjarne Stroustrup. *The C++ Programming Language*. 4th. Addison-Wesley Professional, 2013.
- [20] Matteo Turisini, Giorgio Amati e Mirko Cestari. «LEONARDO: A Pan-European Pre-Exascale Supercomputer for HPC and AI applications». In: *Journal of Large-Scale Research Facilities JLSRF* 9.1 (2024), A186. DOI: 10.17815/jlsrf-8-186. URL: <https://jlsrf.org/index.php/lsf/article/view/186>.
- [21] TOP500 Project. *TOP500 Supercomputer*. <https://top500.org>. 2026.

-
- [22] EuroHPC Joint Undertaking. *Leonardo is inaugurated: Europe welcomes a new world-leading supercomputer*. Comunicato stampa online. Nov. 2022. URL: https://www.eurohpc-ju.europa.eu/leonardo-inaugurated-europe-welcomes-new-world-leading-supercomputer-2022-11-24_en.
- [23] NVIDIA Corporation. *NVIDIA Collective Communication Library (NCCL)*. <https://github.com/NVIDIA/ncc1>. 2026.
- [24] Siyuan Shen et al. *ATLAHS: An Application-centric Network Simulator Toolchain for AI, HPC, and Distributed Storage*. 2025. arXiv: 2505.08936 [cs.DC]. URL: <https://arxiv.org/abs/2505.08936>.

Ringraziamenti

*"Voglio esprimere la mia più profonda gratitudine al mio relatore, il **Prof. Daniele De Sensi**, per la disponibilità, la guida autorevole e il supporto che mi ha offerto durante tutto lo sviluppo del progetto e della stesura di questo elaborato.*

*Un ringraziamento immenso va alla mia **famiglia** per avermi sempre sostenuto, per la fiducia che ha riposto in me e per avermi garantito la serenità necessaria a completare questo percorso di studi.*

*Ringrazio di cuore **Federica**, per essere stata una presenza costante e preziosa, per aver condiviso con me ogni momento di questo percorso, ascoltando i miei dubbi e dandomi la carica e l'ispirazione nei momenti più intensi.*

*Infine, desidero ringraziare il mio amico e collega **Federico**, con cui ho condiviso questi anni di corso. Grazie per le lunghe giornate di studio, i progetti affrontati insieme e i momenti di svago notturni con le nostre immancabili partite serali."*

Roma, 8 luglio 2026